جامعة القدس
Al-Quds University

Faculty of Engineering

Department of Computer Engineering

Software Project

Project Name:

(PureSkin Clinic)

By:

Mohammad Abu Ismail-22212000

Othman Othman-22212228

Supervisor:

Dr. Safa Nasser Eldin

This software project is submitted in partial fulfillment of the requirements for the degree of BSc in Computer Engineering from Al-Quds University.

Dedication

To our beloved parents, whose endless love, patience, and encouragement have been the foundation of every achievement in our lives.

To our families and friends, who supported us throughout this journey with constant motivation and belief in our abilities.

To our teachers and mentors at Al-Quds University, who generously shared their knowledge and guided us toward becoming engineers.

We dedicate this work to all of you, with our deepest gratitude.

Acknowledgment

First and foremost, we thank God for granting us the strength, patience, and determination to complete this project.

We would like to express our sincere gratitude to our supervisor, Safa Nasser Eldin, for her invaluable guidance, continuous support, and constructive feedback throughout the development of this project. Her insights greatly shaped the quality of this work.

We also extend our appreciation to the Faculty of Engineering and the Department of Computer Engineering at Al-Quds University for providing the knowledge, resources, and environment that made this project possible.

Finally, we are deeply grateful to our families and friends for their unwavering encouragement and support during our studies.

Abstract

PureSkin Clinic is a full-stack, role-based web application developed to digitize and unify the operations of a dermatology and aesthetic clinic. Traditional clinic management—based on phone bookings, paper records, and separate billing tools—suffers from scheduling conflicts, slow record retrieval, and disconnected sales and clinical data. This project addresses these problems by integrating appointment management, patient medical records, doctor–patient messaging, product e-commerce, and administrative control into a single platform.

The system is implemented as a React single-page application communicating with a Node.js and Express REST API, backed by a MySQL relational database. Security is enforced through JSON Web Token (JWT) authentication and role-based access control for the Patient, Doctor, and Administrator roles, while email notifications are delivered through an SMTP service and online payments are processed through Stripe Checkout alongside a Cash on Delivery option. The database schema is managed through a versioned migration system that guarantees controlled and repeatable evolution.

The application is fully implemented and deployed in production at https://pureclinic.online, with its API served from https://api.pureclinic.online/api. It was validated through manual functional and integration testing covering authentication, appointment booking, the completion gate, checkout with stock guarding and webhook idempotency, and review moderation. The results confirm that the core workflows for all three roles operate correctly and that the system's security and validation rules are enforced. The report concludes with recommendations for future work, most notably the addition of an automated test suite and continuous-integration testing.

Table of Contents

List of Figures

List of Tables

Chapter One: Introduction

1.1 Overview

The increasing adoption of digital technologies in healthcare has created demand for integrated platforms that streamline clinic operations and improve patient experience. Traditional clinic management — relying on phone-based booking, paper records, and disconnected billing — is no longer sufficient for modern service expectations.

PureSkin Clinic is a full-stack, role-based web application developed to address these challenges. The system unifies appointment management, patient medical records, doctor–patient communication, product e-commerce, and administrative control into a single platform. It is publicly deployed at https://pureclinic.online and serves patients, doctors, and clinic administrators through dedicated, access-controlled portals.

1.2 Project Idea Description

The project consolidates core clinic functions — fragmented across phone calls, paper records, and separate billing tools — into a centralized web platform. Patients can register with email verification, book appointments, manage their medical profiles, shop for clinic products, track orders, and message their doctors. Doctors can manage their appointment queues, access patient records, and document treatment sessions. Administrators manage users, products, inventory, orders, and reviews through a permission-gated dashboard.

The system is built on a modern full-stack architecture: a React SPA frontend, a Node.js/Express REST API backend, and a MySQL relational database. External integrations include Stripe for card payments and SMTP for automated email notifications. The application is fully deployed and live at https://pureclinic.online, with the backend API accessible at https://api.pureclinic.online/api.

1.3 Problem Statement

Small-to-medium clinics commonly operate through disconnected and manual workflows, resulting in the following challenges:

- Appointment scheduling via phone or paper logs leads to conflicts and double-bookings.
- Patient medical records stored in physical files are slow to retrieve and prone to error.

- No integrated communication channel exists between patients and clinical staff.
- Product sales are managed separately from clinical operations, causing inventory inconsistencies.
- Patients and clinic management lack visibility into treatment costs and purchase history.

PureSkin Clinic addresses these problems by providing a centralized, secure, and role-based digital platform that standardizes clinical workflows and patient-facing services.

1.4 Project Objectives

The main objectives of this project are:

- Provide a secure, role-based web platform for patients, doctors, and administrators.
- Enable verified patient registration through email-based OTP account activation.
- Implement appointment lifecycle management: booking, confirmation, completion, and cancellation.
- Support treatment documentation and medical record management by authorized doctors.
- Establish in-app messaging between patients and doctors.
- Integrate a product catalog with cart, checkout, and order tracking supporting COD and Stripe payments.
- Provide administrative dashboards with financial and operational reporting.
- Deploy the system as a live production application at https://pureclinic.online.

1.5 Project Benefits

For Patients:

- Self-service appointment booking and order tracking accessible from any device.
- Centralized access to medical profiles, billing summaries, and purchase history.
- Direct, documented messaging channel with treating doctors.

For Doctors:

- Structured appointment queue with clear status management.
- Instant access to patient clinical histories and treatment records.
- Integrated treatment session documentation linked to appointments and billing.

For the Clinic (Administrators):

- Reduced manual workload across scheduling, records, inventory, and order management.
- Granular role-based permissions for admin staff with moderation workflows.

- Unified clinical and e-commerce revenue channel in a single deployed platform.

1.6 Project Methodology

The system was developed using an iterative and incremental approach, dividing the project into phases: requirements analysis, system design, implementation, and testing. Features were built, reviewed, and refined incrementally — evidenced by 19 versioned database migration files representing distinct development increments.

The backend follows a layered architecture (routes → controllers → models → services), ensuring separation of concerns. The frontend uses a React component-based design with shared API services. Database schema evolution is managed through a versioned migration runner executed at server startup. External services (Stripe, SMTP) were integrated incrementally with environment-based configuration.

1.7 Research Plan

The following table outlines the project development phases and their estimated durations:

Phase	Tasks	Duration
1 — Requirements Analysis	Role definition, system scope, literature review	Weeks 1–3
2 — System Design	Architecture, database schema, UI wireframes	Weeks 4–6
3 — Backend Implementation	API, authentication, core modules	Weeks 7–11
4 — Frontend Implementation	SPA pages, routing, API integration	Weeks 9–13
5 — Integration and Testing	End-to-end testing, bug fixes, Stripe/SMTP	Weeks 14–16
6 — Deployment and Documentation	CI/CD pipeline, domain setup, final report	Weeks 16–18

1.8 Project Constraints

The following constraints apply to this project:

- The system requires three external services to be operational: MySQL, an SMTP provider, and Stripe for card payments.
- Appointment scheduling uses fixed predefined time slots; dynamic slot generation is out of scope.
- The monolithic client-server architecture may limit independent scaling under high load.
- No automated test suite is implemented; testing was performed manually.
- The platform is web-only; native Android/iOS applications are out of scope.
- Multi-clinic or multi-tenant support is not included in the current version.

1.9 Literature Review

Several existing systems address parts of the problem domain targeted by PureSkin Clinic. SimplePractice and Cliniko are commercial clinic management platforms offering appointment scheduling, patient records, and billing. However, both are subscription-based SaaS products without source-level customization and without integrated e-commerce. Fresha targets wellness booking through a marketplace model but lacks clinical record management.

Academic literature on healthcare information systems emphasizes the importance of role-based access control (RBAC) for protecting sensitive patient data [1], the integration of electronic health records with appointment management [2], transactional database design for data consistency in clinical environments [3], and secure authentication for patient portal onboarding [4].

PureSkin Clinic fills the gap between general clinic management tools and dermatology-specific workflows by combining appointment management, treatment documentation, messaging, and an integrated product store in a single, live-deployed platform.

1.10 Conclusion and Recommendations

This chapter introduced PureSkin Clinic, defined the problem it addresses, and outlined its objectives, benefits, methodology, and constraints. The system is fully implemented and deployed at https://pureclinic.online, demonstrating that the proposed solution is technically feasible and operational beyond the academic context.

Subsequent chapters present the system analysis (Chapter Two), functional and non-functional requirements (Chapter Three), system design (Chapter Four), implementation details (Chapter Five), and testing strategy (Chapter Six). Future work should prioritize automated testing, security hardening, and scalable deployment infrastructure.

Chapter Two: System Analysis

2.1 Introduction

This chapter analyzes the PureSkin Clinic system from an engineering perspective, covering the organizational context, implementation alternatives, the proposed system architecture, technical and hardware requirements, development costs, feasibility assessment, and project management approach. The system is a deployed web application combining clinic management and e-commerce, accessible at https://pureclinic.online.

2.2 A Brief History of the Organization

PureSkin Clinic is a dermatology and aesthetic clinic represented as the subject of this project. The clinic's brand identity is established throughout the system UI under the name "PureSkin Clinic" and its services are organized into three categories: Hair Care, Laser Treatments, and Skin Care. These are reflected in the appointment treatment categories (hair, skin, body) and the product store catalog.

The system is fully deployed as a live production application at https://pureclinic.online, with the REST API served at https://api.pureclinic.online/api. Both services are managed through automated GitHub Actions deployment to a Linux server.

2.3 Project Implementation Options

Three implementation approaches were evaluated before selecting the final development strategy:

Option	Advantages	Disadvantages	Suitability
A — Standalone Desktop App	Offline capability; direct hardware access	OS-specific builds; no remote patient access; no public store URL	Low
B — Native Mobile App	Push notifications; app-store presence	Two separate codebases; store review cycles; poor admin desktop UX	Medium (future)
C — Web	Single deployment for	Requires internet;	High

Option	Advantages	Disadvantages	Suitability
Application (Chosen)	all roles; bilingual support; CI/CD integration; Stripe redirect flow	HTTPS/CORS configuration required	

The web application approach was selected because it enables multi-role access from any device through a single deployment, supports bilingual EN/AR interfaces, integrates directly with Stripe Checkout browser redirects, and allows automated deployment to the live domain pureclinic.online via GitHub Actions.

2.4 The Proposed System

PureSkin Clinic is a full-stack, role-based web application that unifies clinic management and online product retail in a single platform. The system is live at https://pureclinic.online and serves three user roles through dedicated portals:

Role	Key Capabilities
Patient	Register with email verification; book and track appointments; manage medical profile; shop products; checkout via COD or Stripe; message doctors; view billing summary
Doctor	Manage appointment queue and status; access patient clinical records; record treatment sessions; message patients
Admin	Manage users, doctors, products, inventory, and orders; view financial reports; moderate reviews; assign staff sub-roles
Public / Guest	Browse clinic services and product catalog; register or login

Patient Journey

Step	Action	API Endpoint
1	Register account	POST /api/auth/register

Step	Action	API Endpoint
2	Verify email with OTP	POST /api/auth/verify-email
3	Login	POST /api/auth/login
4	Book appointment	POST /api/appointments
5	Doctor confirms	PUT /api/appointments/:id/status
6	Doctor records treatment	POST /api/doctor/appointments/:id/treatment
7	Patient submits review	POST /api/reviews

E-Commerce Journey

Step	Action	API / Service
1	Browse products	GET /api/products
2	Add to cart	localStorage (client-side)
3	Preview order totals	POST /api/orders/checkout-preview
4a	Place COD order	POST /api/orders/create-checkout-session
4b	Pay by card/Apple Pay	Stripe Checkout → POST /api/webhooks/stripe
5	Track order history	GET /api/orders/my

2.5 System Requirements

Server Hardware

Item	Minimum	Recommended
CPU	1 vCPU	2+ vCPU
RAM	2 GB	4 GB

Item	Minimum	Recommended
Storage	20 GB SSD	40 GB SSD
OS	Linux (Ubuntu)	Linux Ubuntu 20.04+
Network	Stable internet (Stripe + SMTP outbound)	Same + reverse proxy

Software Requirements

Component	Version	Source
Node.js	20 (LTS)	.github/workflows/deploy-linux.yml
React + Vite	19.2.0 + 7.3.1	client/package.json
Express	4.19.2	server/package.json
MySQL	Required (version not pinned)	mysql2 driver
Stripe SDK	16.0.0	server/package.json
Nodemailer	8.0.5	server/package.json

Key Environment Variables

Category	Variables	Required?
Database	DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	Yes
Authentication	JWT_SECRET, JWT_EXPIRES_IN	Yes
Stripe	STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET	Yes (card payments)
Email (SMTP)	SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS,	Yes (email features)

Category	Variables	Required?
	MAIL_FROM	
Frontend (build)	VITE_API_BASE_URL, VITE_STRIPE_PUBLISHABLE_KEY	Yes (production)
Server	PORT, CLIENT_ORIGIN, NODE_ENV	Yes

2.6 Project Development and Cost

2.6.1 Hardware

Item	Units	Unit Cost	Available	Total
Developer laptop/PC (i5+, 8GB RAM, 256GB SSD)	2	$800	Yes	$1,600
Internet connection	1	$50/month	Yes	$300 (6 months)
Linux VPS (production)	1	$20/month	Yes	$120 (6 months)
Total				$2,020

2.6.2 Software

Item	Units	Unit Cost	Available	Total
Node.js, npm, React, Vite, Tailwind CSS	1	$0 (open-source)	Yes	$0
Express, MySQL2, JWT, bcrypt, Nodemailer, Stripe SDK	1	$0 (open-source)	Yes	$0
Git, GitHub, VS Code, ESLint	1	$0 (open-source)	Yes	$0
Domain (pureclinic.online)	1	$15/year	Yes	$15
SSL/TLS (Let's Encrypt)	1	$0	Yes	$0

Item	Units	Unit Cost	Available	Total
Total				$15

2.6.3 Humans

Member	Number	Cost/Month	Available	Total (6 months)
Full-stack developers (students)	2	$250	Yes	$3,000
Academic supervisor	1	$400	Yes	$2,400
Total				$5,400

2.6.4 Books and References

Resource	Type	Cost	Total
React, Node.js, MySQL, Stripe official docs	Online documentation	$0	$0
MDN Web Docs, GitHub Docs	Online documentation	$0	$0
Total			$0

2.6.5 Others

Item	Cost	Notes
Stripe transaction fees	2.9% + $0.30/transaction	Applied on successful card payments only
Report printing	$30	Academic submission
Total (fixed)	$30	

2.6.6 Total Cost Summary

Type	Total
Hardware	$2,020
Software (open-source)	$0
Software (domain)	$15
Human resources	$5,400
Books and references	$0
Others	$30
Grand Total	$7,465

2.7 Cost Benefits

For the Development Team

- Gained full-stack experience: React, Node.js, Express, MySQL, JWT, and REST API design.
- Implemented real-world integrations: Stripe webhook handling, SMTP email, and node-cron scheduling.
- Practiced CI/CD with GitHub Actions, SSH deployment, and systemd service management.
- Developed i18n and RTL support using react-i18next for a bilingual EN/AR interface.
- Delivered a live, publicly accessible application at https://pureclinic.online as a portfolio artifact.

For the Clinic and Society

- Eliminates phone-based booking — patients self-schedule online at any time.
- Centralizes patient records in a secure, role-restricted database.
- Creates an online revenue channel with COD and Stripe card payment support.
- Automated reminder emails reduce patient no-shows.
- Bilingual interface increases accessibility for Arabic-speaking patients.

2.8 Feasibility Study

2.8.1 Economic Feasibility

The project is economically feasible. The full technology stack is open-source with no licensing fees. Primary costs are developer time and hosting. A Linux VPS costs approximately $20/month, the domain pureclinic.online costs $15/year, and SSL is free via Let's Encrypt. The estimated total project cost of $7,465 is appropriate for a two-developer academic project.

2.8.2 Technical Feasibility

All technologies used are mature, well-documented, and widely adopted in production environments. Technical feasibility is confirmed by the live deployment at https://pureclinic.online, 19 applied database migrations, a functional GitHub Actions CI/CD pipeline, and a verified Stripe webhook integration.

Risk	Mitigation
No automated test suite	Manual testing performed; Jest/Supertest planned for future iterations
JWT_SECRET fallback value	Strong secret enforced through GitHub Secrets in production pipeline
Single-server deployment	Acceptable for academic scope; systemd auto-restart configured

2.8.3 Legal Feasibility

The system collects personal and medical data including patient name, email, phone, date of birth, address, skin type, allergies, medical history, and treatment billing. Card data is processed exclusively by Stripe Checkout and is never stored in the local database, reducing PCI DSS compliance scope. Webhook events are verified using Stripe signature authentication. All dependencies use MIT/ISC-compatible open-source licenses.

2.8.4 Schedule Feasibility

Development progress is traceable through 19 numbered migration files, each representing a distinct feature increment. The estimated development timeline for two developers is 18–24 weeks.

Phase	Tasks	Duration
1 — Core Schema and Auth	Database design, Users, JWT, patient registration (001–002)	Weeks 1–2
2 — Appointments and Doctor Tools	Booking, status workflow, treatment sessions (003–009)	Weeks 3–4
3 — Store and Orders	Product catalog, cart, COD checkout (010–011)	Weeks 5–7
4 — Stripe and Admin Panel	Stripe checkout, webhooks, admin permissions (012–015)	Weeks 8–9
5 — Reviews, Email, Reminders	Review moderation, email verification, cron (016–018)	Weeks 10–11
6 — Deployment and Polish	CI/CD pipeline, domain setup, i18n, report (019)	Weeks 12–13

2.9 Project Added Values

Added Value	Description
Clinic + E-Commerce Integration	Appointment management and product store in a single deployed platform
Bilingual EN/AR Interface	Full English and Arabic support with RTL layout switching
Granular Admin Permissions	Staff sub-roles (store_manager, order_manager, etc.) with scoped access
Dual Payment Methods	Cash-on-delivery and Stripe card/Apple Pay with webhook fulfillment

Added Value	Description
Automated Reminders	Cron job sends appointment reminder emails without manual intervention
Live Production Deployment	Publicly accessible at https://pureclinic.online via CI/CD pipeline

2.10 Project Management

The project was managed using GitHub for version control and collaboration. The codebase is organized into three directories (client/, server/, database/) to minimize merge conflicts between frontend and backend development. All pushes to the main branch trigger automated deployment via GitHub Actions.

CI/CD Pipeline Summary

Stage	Details
Trigger	Push to main branch or manual dispatch
Frontend Build	npm ci → npm run build with VITE_API_BASE_URL and VITE_STRIPE_PUBLISHABLE_KEY
Frontend Deploy	SCP dist/ to server; systemd service on port 9073 (serve --single)
Backend Deploy	SCP server/; write .env; npm ci --omit=dev; systemd service on port 9072 (node server.js)
Health Checks	curl /api/health (port 9072) and frontend (port 9073) after deploy
Secrets	SERVER_HOST, SERVER_USER, SSH_PRIVATE_KEY, VITE_STRIPE_PUBLISHABLE_KEY

Database change management was handled through a custom JavaScript migration runner (server/config/db.js) that tracks applied migrations in a Migrations table and automatically executes new migration files on server startup. A total of 19 migrations were applied incrementally across the development lifecycle.

2.11 Conclusion and Recommendations

This chapter confirmed that PureSkin Clinic is technically sound, economically viable, and legally compliant within its current scope. The web application approach was the correct implementation choice, enabling multi-role access, bilingual support, Stripe integration, and automated deployment to the live domain https://pureclinic.online.

The following recommendations are made for future development:

1. Implement an automated test suite using Jest and Supertest.
2. Enforce a strong JWT_SECRET in all production environments.
3. Develop a formal privacy policy addressing the personal and medical data stored.
4. Complete the forgot-password feature referenced in the UI localization files.
5. Migrate product image storage from the local filesystem to a cloud storage service.

Chapter Three presents the complete functional and non-functional requirements of the system.

Chapter Three: System Functional Requirements

3.1 Introduction

This chapter defines the functional and non-functional requirements of PureSkin Clinic. It identifies the system actors, presents use-case diagrams, and describes each functional requirement using structured tables. Non-functional requirements covering security, performance, reliability, and usability are also included.

3.2 Use-Case Diagrams

The PureSkin Clinic system has four primary actors — Guest, Patient, Doctor, and Administrator — and one external system actor (Stripe). The following diagrams illustrate the use cases available to each actor.

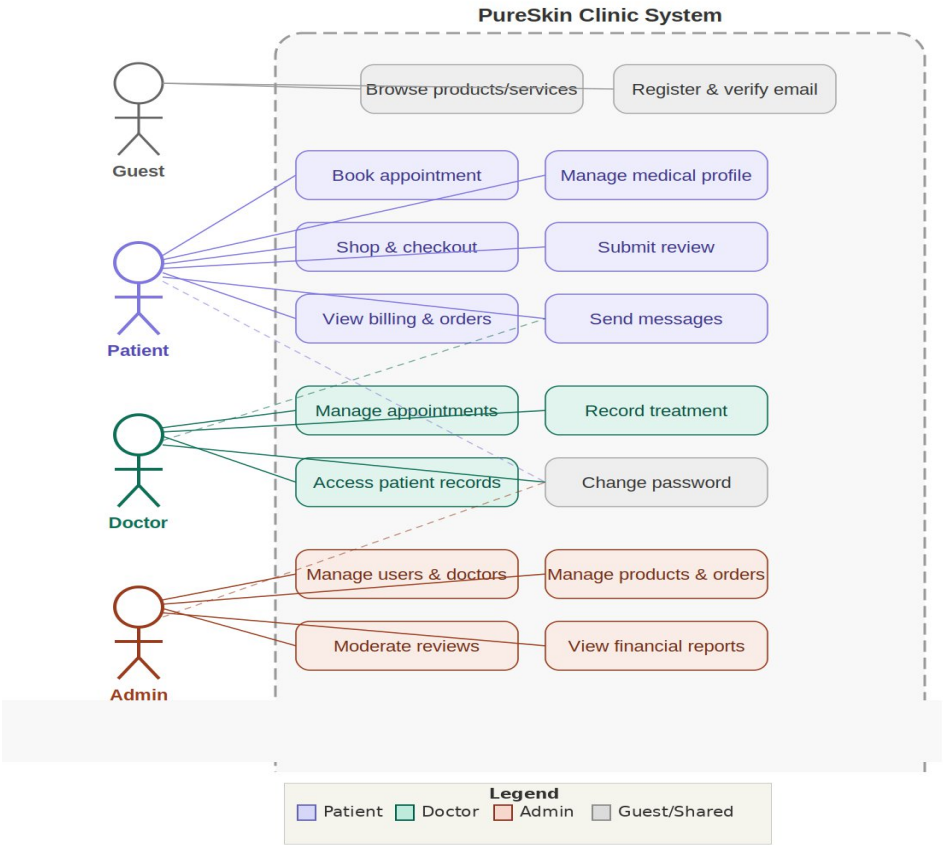

Figure 3.1: PureSkin Clinic System — Overall Use-Case Diagram

3.3 Functional Requirements Description

The following tables describe each functional requirement derived from the implemented API endpoints, controllers, and frontend workflows.

Table 3.1: FR-01 – Patient Registration and Email Verification

Requirement ID	FR-01
Requirement Name	Patient Registration and Email Verification
Description	The system shall allow guests to register as patients and activate their accounts through email OTP verification.
Actor	Guest
Input	Name, email, password; 6-digit OTP code
Source	RegisterPage.jsx → POST /api/auth/register; POST /api/auth/verify-email
Pre-condition	Email not already registered
Series of Operations	1. Guest submits registration form. 2. System validates input, hashes password (bcrypt), creates Users + Patients rows. 3. System sends 6-digit OTP via email. 4. Guest submits OTP. 5. System marks email_verified = true and issues JWT.
Result	Activated patient account; JWT token issued
Destination	/patient/medical-profile
Other Scenarios / Errors	409 – Email already in use; 400 – Invalid/expired OTP; 429 – Resend too soon; 503 – SMTP failure (account created); 422 – Validation errors

Table 3.2: FR-02 – Role-Based Authentication and Authorization

Requirement ID	FR-02

Requirement Name	Role-Based Authentication and Authorization
Description	The system shall authenticate users and restrict access to routes and endpoints based on role (patient/doctor/admin) and granular admin permissions.
Actor	Patient, Doctor, Administrator
Input	Email, password; JWT Bearer token in subsequent requests
Source	LoginPage.jsx → POST /api/auth/login; middleware/auth.js, role.js, authorize.js
Pre-condition	Valid account; patient must be email-verified
Series of Operations	1. User submits credentials. 2. System validates and bcrypt-compares password. 3. JWT issued (signed with JWT_SECRET). 4. Frontend stores token. 5. Protected routes verify token and enforce role/permission.
Result	JWT token; redirect to role-specific dashboard
Destination	/patient/dashboard, /doctor/dashboard, or /admin/dashboard
Other Scenarios / Errors	401 – Invalid credentials; 403 – Patient not verified (EMAIL_NOT_VERIFIED); 403 – Insufficient admin permission

Table 3.3: FR-03 – Patient Appointment Booking

Requirement ID	FR-03
Requirement Name	Patient Appointment Booking
Description	Authenticated patients shall book appointments by selecting a doctor, date, available time slot, and treatment category.
Actor	Patient
Input	Doctor ID, appointment date, time (HH:MM), treatment category (hair/skin/body)

Source	AppointmentsPage.jsx → GET /api/appointments/doctors, GET /api/appointments/slots, POST /api/appointments/
Pre-condition	Authenticated patient; selected time slot not already booked
Series of Operations	1. Patient fetches doctor list. 2. Patient selects doctor and date; system returns available hourly slots (09:00–17:00). 3. Patient submits booking. 4. System validates and creates appointment (status: pending). 5. Booking notification email sent.
Result	201 – Appointment created with status = pending
Destination	/patient/appointments
Other Scenarios / Errors	409 – Slot already booked; 422 – Missing/invalid fields; 404 – Patient profile not found

Table 3.4: FR-04 – Appointment Status Management

Requirement ID	FR-04
Requirement Name	Doctor Appointment Lifecycle Management
Description	Doctors shall manage appointment status through a defined state machine: pending → confirmed or cancelled; confirmed → completed or cancelled.
Actor	Doctor
Input	Appointment ID, target status
Source	DoctorAppointmentsPage.jsx → PUT /api/appointments/:id/status
Pre-condition	Doctor owns the appointment; transition is valid
Series of Operations	1. Doctor selects transition action. 2. System verifies ownership and validates transition. 3. For completion: checks treatment evidence gate (price > 0, paid > 0, or notes ≥ 3 chars). 4. System updates status. 5. Confirmation email sent on confirmed.
Result	200 – Updated appointment with new status

Destination	/doctor/appointments
Other Scenarios / Errors	400 – Invalid/forbidden transition; 400 – Completion blocked (no evidence); 403/404 – Not doctor's appointment

Table 3.5: FR-05 – Treatment Session Recording

Requirement ID	FR-05
Requirement Name	Doctor Treatment Session Recording
Description	Doctors shall record or update treatment sessions linked to appointments, including clinical notes, session price, and amount paid.
Actor	Doctor
Input	Appointment ID; optional: sessionPrice, amountPaid, notes, completeAppointment flag
Source	Doctor UI → POST /api/doctor/appointments/:appointmentId/treatment
Pre-condition	Doctor owns the appointment
Series of Operations	1. Doctor enters treatment details. 2. System creates or updates TreatmentSessions record. 3. If completeAppointment = true and evidence passes: appointment set to completed.
Result	200 – { treatment, appointment? }
Destination	/doctor/appointments, /doctor/patients/:patientId
Other Scenarios / Errors	404 – Appointment not found; 403 – Not doctor's appointment; 400 – Completion without evidence

Table 3.6: FR-06 – Patient Profile and Medical Record Management

Requirement ID	FR-06

Requirement Name	Patient Self-Managed Profile and Medical Record
Description	Authenticated patients shall view and update their demographic profile and medical record (skin type, allergies, medications, history).
Actor	Patient
Input	Profile: phone, date of birth, address. Medical: skin type, complaints, allergies, medications, pregnancy status, notes
Source	MedicalProfilePage.jsx → GET/PUT /api/patients/me, PUT /api/patients/me/medical-record
Pre-condition	Authenticated patient account exists
Series of Operations	1. System returns existing profile and medical record. 2. Patient edits and submits. 3. System validates and updates Patients and MedicalRecords tables.
Result	200 – Updated profile and/or medical record
Destination	/patient/medical-profile
Other Scenarios / Errors	422 – Invalid date format; 404 – Patient not found

Table 3.7: FR-07 – Patient–Doctor Messaging

Requirement ID	FR-07
Requirement Name	Secure In-App Messaging
Description	The system shall support bidirectional message exchange between patients and doctors with conversation history.
Actor	Patient, Doctor
Input	Receiver user ID, message text

Source	MessagesPage.jsx → GET /api/messages/conversations, GET /api/messages/with/:userId, POST /api/messages
Pre-condition	Sender is authenticated; receiver exists
Series of Operations	1. System returns conversation list. 2. User selects thread. 3. User sends message. 4. System stores message with sender_id and receiver_id.
Result	201 – Message stored; 200 – Thread returned
Destination	/patient/messages, /doctor/messages
Other Scenarios / Errors	400 – Empty message or missing receiver; 401 – Unauthenticated

Table 3.8: FR-08 – Product Browsing and Cart Management

Requirement ID	FR-08
Requirement Name	Product Browsing and Cart Management
Description	Users shall browse active products publicly. Authenticated patients shall manage a shopping cart stored in browser localStorage.
Actor	Guest (browse), Patient (cart)
Input	Search query, category filter; cart: productId, quantity
Source	StorePage.jsx, CartPage.jsx → GET /api/products, GET /api/products/:id
Pre-condition	Products must be active (is_active = true); patient must be authenticated for /store and /cart
Series of Operations	1. User browses product catalog. 2. Patient adds to cart (localStorage: pureskin_cart). 3. Patient adjusts quantities. 4. System displays totals.
Result	200 – Product list/detail; cart state persisted locally

Destination	/products, /store, /cart
Other Scenarios / Errors	404 – Product inactive or not found; guest redirected to /login for cart access

Table 3.9: FR-09 – Checkout and Order Placement

Requirement ID	FR-09
Requirement Name	Multi-Method Order Processing
Description	Patients shall complete purchases via Cash-on-Delivery or Stripe card/Apple Pay. Server-side pricing is authoritative. Stock decremented atomically on order confirmation.
Actor	Patient
Input	items [{productId, quantity}], paymentMethod (cash_on_delivery/card/apple_pay), phone, address
Source	CartPage.jsx → POST /api/orders/checkout-preview, POST /api/orders/create-checkout-session
Pre-condition	Authenticated patient; items active and in stock; phone and address provided
Series of Operations	1. Server validates items and computes trusted totals (checkout-preview). 2. Patient submits order. 3. COD: transaction decrements stock + creates Order (status: pending). 4. Card: creates Order (awaiting_payment) + Stripe session URL returned. 5. Stripe webhook finalizes card order (FR-10).
Result	COD: 201 – Order confirmed. Card: 200 – Stripe checkout URL.
Destination	/orders
Other Scenarios / Errors	409 – Insufficient stock; 400 – Invalid payment method or missing fields; 503 – Stripe not configured

Table 3.10: FR-10 – Stripe Webhook Payment Fulfillment

Requirement ID	FR-10
Requirement Name	Stripe Webhook Order Fulfillment
Description	The system shall idempotently process Stripe checkout.session.completed events: verify signature, check duplicates, decrement stock, and mark order as paid.
Actor	Stripe (external system)
Input	Raw Stripe webhook payload; stripe-signature header
Source	POST /api/webhooks/stripe → stripeWebhookController.js
Pre-condition	Valid Stripe signature; order exists in awaiting_payment status
Series of Operations	1. Verify Stripe signature. 2. Check StripeWebhookEvents for duplicate event. 3. Lock order row and validate payment amount. 4. Transaction: decrement stock + set payment_status = paid. 5. Record event (idempotency). 6. Send confirmation email.
Result	200 – { received: true }; order fulfilled
Destination	/orders?session_id=...
Other Scenarios / Errors	400 – Invalid signature; 200 – Duplicate event (no-op); 400 – Amount mismatch; 404 – Order not found

Table 3.11: FR-11 – Admin User and Doctor Management

Requirement ID	FR-11
Requirement Name	Admin User and Doctor Account Management
Description	Authorized administrators shall manage user accounts, assign roles, and create/delete doctor profiles.
Actor	Administrator

Input	User update fields (name, role); doctor data (name, email, specialization, password)
Source	UsersPage.jsx, DoctorsPage.jsx → GET/PATCH/DELETE /api/admin/users/:id, /api/admin/doctors/:id
Pre-condition	Admin with ADMIN_MANAGE_USERS or ADMIN_MANAGE_DOCTORS permission
Series of Operations	1. Admin searches/filters records. 2. Admin creates, updates, or deletes record. 3. System validates permissions and applies changes.
Result	Updated user/doctor records
Destination	/admin/users, /admin/doctors
Other Scenarios / Errors	403 – Insufficient permissions; 409 – Duplicate email; 400 – Invalid role; 404 – User not found

Table 3.12: FR-12 – Admin Product and Inventory Management

Requirement ID	FR-12
Requirement Name	Admin Product Catalog and Stock Management
Description	Authorized administrators shall create, update, and delete products with image upload, and perform auditable stock adjustments.
Actor	Administrator
Input	Product fields (name, category, price, discount, stock, image file); stock delta and reason
Source	ProductsPage.jsx → POST/PUT/DELETE /api/admin/products/:id, POST /api/admin/products/:id/stock-adjustment
Pre-condition	Admin with PRODUCTS_* or INVENTORY_* permission
Series of Operations	1. Admin creates/updates product (multer image upload to server/uploads/). 2. Admin applies stock adjustment. 3. Transaction

	updates Products.stock + inserts ProductStockAdjustments record.
Result	Updated product; auditable stock log entry created
Destination	/admin/products
Other Scenarios / Errors	404 – Product not found; 400 – Cannot delete product with existing orders; 422 – Validation errors

Table 3.13: FR-13 – Administrative Order Management

Requirement ID	FR-13
Requirement Name	Administrative Order Operations
Description	Authorized administrators shall view all orders, update statuses, add notes, and cancel orders with automatic inventory restoration.
Actor	Administrator
Input	Order ID, target status, payment status, admin notes
Source	AdminOrdersPage.jsx → GET /api/admin/orders, PATCH /api/admin/orders/:orderId/status, /notes, /payment-status
Pre-condition	Admin with ORDERS_* permission; order exists
Series of Operations	1. Admin filters and views orders. 2. Admin updates status, payment status, or notes. 3. On cancellation: transaction restores stock if previously deducted.
Result	200 – Updated order; inventory restored on eligible cancellation
Destination	/admin/orders
Other Scenarios / Errors	400 – Invalid transition or already cancelled; 403 – Insufficient permissions; 404 – Order not found

Table 3.14: FR-14 – Review Submission and Moderation

Requirement ID	FR-14
Requirement Name	Patient Review Submission and Admin Moderation
Description	Eligible patients shall submit one review per completed appointment. Administrators shall moderate review visibility through a defined status lifecycle.
Actor	Patient (submission), Administrator (moderation)
Input	Patient: appointmentId, rating (1–5), comment (3–2000 chars). Admin: review ID, target status
Source	AppointmentsPage.jsx → POST /api/reviews; Admin → PATCH /api/reviews/:id/status
Pre-condition	Patient: appointment completed and belongs to patient; one review per appointment. Admin: ADMIN_MANAGE_REVIEWS permission
Series of Operations	1. Patient submits review for completed appointment. 2. System validates eligibility; stores review (status: pending). 3. Admin applies moderation action (approve/reject/hide/flag). 4. Public GET /api/reviews returns approved reviews only.
Result	201 – Review created; moderation updates status
Destination	/patient/appointments; public landing page
Other Scenarios / Errors	409 – Duplicate review; 400 – Appointment not completed; 403 – Wrong patient or insufficient permission

Table 3.15: FR-15 – Automated Email Notifications and Reminders

Requirement ID	FR-15
Requirement Name	Automated Notifications and Reminder Service
Description	The system shall send transactional emails on key events and run a scheduled cron job for appointment reminder emails.

Actor	System (automated); Patient and Doctor as recipients
Input	Trigger events: registration, booking, order confirmation. Cron config: APPOINTMENT_REMINDER_CRON, APPOINTMENT_REMINDER_HOURS_BEFORE
Source	emailNotifications.js, mailer.js, appointmentReminderJob.js
Pre-condition	SMTP configured; APPOINTMENT_REMINDER_ENABLED = true for reminders
Series of Operations	1. Business event triggers email dispatch via Nodemailer/SMTP. 2. Cron job queries upcoming appointments in reminder window. 3. Reminder emails sent; reminder_email_sent_at recorded to prevent re-send.
Result	Email delivered to recipient; duplicates prevented via timestamp
Destination	Patient/doctor email inbox
Other Scenarios / Errors	SMTP failure: error logged, system continues; duplicate reminders prevented by migration 018 timestamp field

3.4 Non-Functional Requirements Description

3.4.1 Security

- All protected endpoints require a valid JWT Bearer token (server/middleware/auth.js).
- Role enforcement via roleMiddleware; granular admin permissions via requirePermission() (authorize.js).
- Passwords stored as bcrypt hashes (genSalt(10)); plaintext storage is prohibited.
- Stripe webhook signature verified using stripe.webhooks.constructEvent() with STRIPE_WEBHOOK_SECRET.
- Server-side input validation enforced via express-validator on all mutating operations (422 responses).
- CORS restricted to whitelisted CLIENT_ORIGIN values (server/app.js).

3.4.2 Performance

The following database indexes support query performance on frequently accessed tables:

Index	Table	Migration Source
idx_appointments_patient_date, idx_appointments_doctor_date	Appointments	001_init_schema.js
idx_messages_participants	Messages	001_init_schema.js
idx_products_category, idx_products_active	Products	001, 011
idx_orders_status_created, idx_orders_payment_status	Orders	001, 010
idx_treatments_appointment	TreatmentSessions	005_add_treatment_sessions.js
idx_reviews_status_created, idx_reviews_patient_created	Reviews	007_add_reviews.js
idx_psa_product, idx_psa_created	ProductStockAdjustments	015_product_stock_adjustments.js

- Connection pool: connectionLimit: 10, waitForConnections: true (server/config/db.js).

3.4.3 Usability

- Role-specific dashboards and navigation for patient, doctor, and admin workflows.
- Inline validation and error feedback on all forms (react-hot-toast).
- Bilingual EN/AR with RTL layout via react-i18next (client/src/locales/).
- Light/dark theme via pureskin_theme cookie and CSS class toggle (SettingsContext.jsx).
- Responsive layouts using Tailwind CSS mobile-first breakpoints.

3.4.4 Reliability

- COD orders, Stripe webhook fulfillment, admin cancellations, and stock adjustments use DB transactions.
- Appointment and order state transitions governed by defined state machines.

- Stripe webhook idempotency via StripeWebhookEvents table (duplicate events ignored).
- Reminder emails deduplicated via reminder_email_sent_at timestamp (migration 018).
- Centralized error handler returns structured JSON without exposing server internals.

3.4.5 Availability

- systemd Restart=always configured for both backend and frontend services in production.
- Health check endpoint: GET /api/health → { status: 'ok', timestamp }.
- Cash-on-Delivery path operates independently of Stripe availability.

3.4.6 Scalability

- Stateless JWT authentication supports horizontal scaling behind a load balancer.
- Database indexes on hot query paths support growth in appointments and orders.
- Frontend static assets served separately from API (pureclinic.online vs api.pureclinic.online).
- Current limitations: single Node.js process; local filesystem for product images; monolithic architecture.

3.4.7 Maintainability

- Layered backend: routes → controllers → models → services → middleware.
- Database changes managed through 19 versioned migration files, auto-applied at startup.
- Frontend organized by domain feature (auth/, patient/, doctor/, admin/, store/).
- Environment config via dotenv loading server/app.env.

3.4.8 Compatibility

- Compatible with modern browsers (Chrome, Firefox, Safari, Edge) on desktop and mobile.
- Stripe API version: 2023-10-16; SDK: ^16.0.0 (server/config/stripe.js).
- React: 19.2.0; Node.js: 20 LTS; Vite: 7.3.1.

3.4.9 Privacy

- Patient medical data accessible only to the patient, treating doctors, and authorized admins.

- Card payment data not stored locally; handled exclusively by Stripe Checkout.
- API responses follow least-privilege principles.

3.4.10 Error Handling

- Consistent HTTP status codes: 400, 401, 403, 404, 409, 422, 500, 503.
- Centralized error middleware catches unhandled exceptions and logs without exposing internals.
- Client-side recoverable error paths: resend OTP, retry payment, inline form validation.

3.5 Conclusion and Recommendations

This chapter presented 15 functional requirements covering all system modules — authentication, appointments, treatment, e-commerce, administration, reviews, and notifications — alongside 10 non-functional quality requirements. All requirements are derived from the implemented codebase and verified against the live deployment at https://pureclinic.online.

It is recommended to prioritize automated test coverage for the highest-risk requirements (FR-04 appointment state machine, FR-09 checkout, FR-10 Stripe webhook), introduce API rate limiting, and enforce strong JWT secrets in all production environments. The next chapter presents the system design including class, sequence, ERD, and activity diagrams.

Chapter Four: System Design

4.1 Introduction

This chapter presents the complete system design of PureSkin Clinic, translating the functional requirements defined in Chapter Three into a concrete architectural blueprint. The design covers the object model (class diagram), the principal interaction flows (four sequence diagrams), the data structure (entity relationship diagram), and the two most important process flows (two activity diagrams).

All diagrams in this chapter are derived directly from the implemented codebase — including server/models/, server/controllers/, server/migrations/, and client/src/pages/ — ensuring full alignment between the design artifacts and the deployed system at https://pureclinic.online.

4.2 Class Diagram

The class diagram maps the thirteen domain entities of PureSkin Clinic to their corresponding MySQL tables. Each class lists its attributes with data types, the data access methods implemented in server/models/, and its associations with multiplicity. The system uses a functional data access pattern — each model exports async functions rather than instantiating class objects.

The diagram (Figure 4.1, next page) reflects the following key design decisions:

- User is the central authentication entity, extended by Patient and Doctor through separate one-to-optional profile tables.
- Appointment is the core clinical entity, linked to both Patient and Doctor, and associated with an optional TreatmentSession and an optional Review.
- Order follows a composition relationship with OrderItem, where each order owns one or more line items referencing Products.
- StripeWebhookEvent is a standalone idempotency table with no foreign key — the link to an order is established logically through the Stripe session ID.
- ProductStockAdjustment records every manual stock change with a reference to the acting admin user, providing a full audit trail.

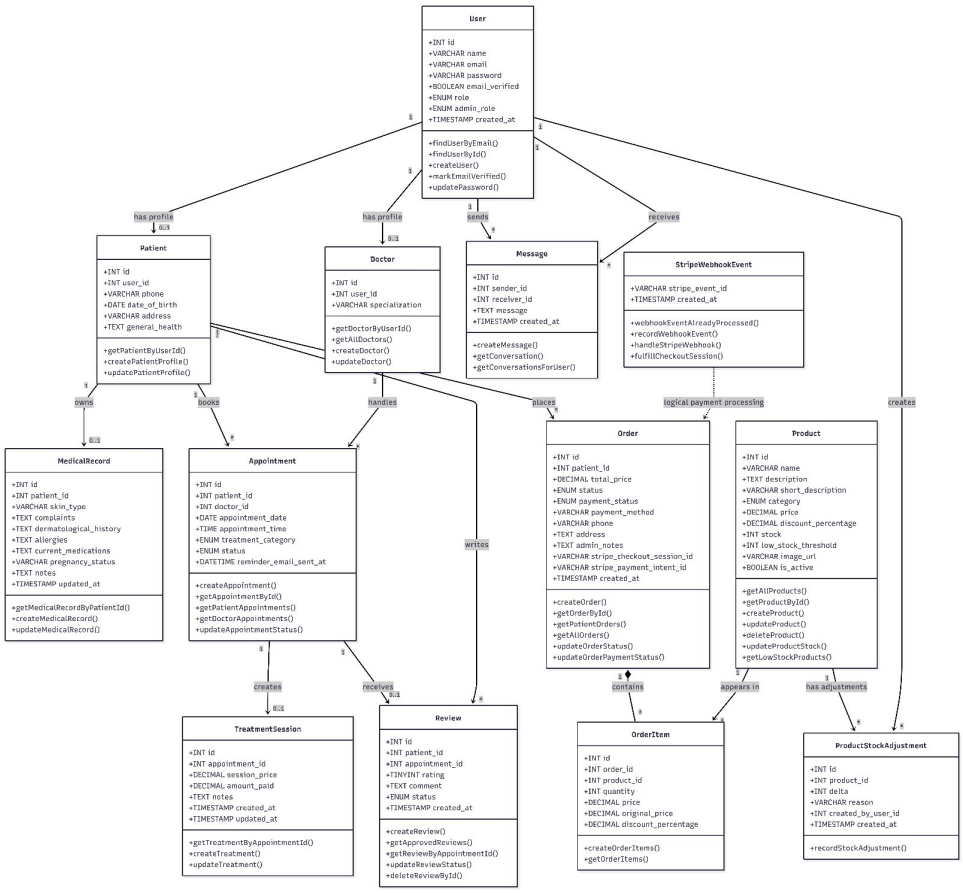

Figure 4.1: PureSkin Clinic — Class Diagram (13 Domain Classes)

4.3 Sequence Diagrams

The following four sequence diagrams trace the principal interaction flows of PureSkin Clinic, showing the messages exchanged between actors and system components (Frontend, Backend API, MySQL database, and external services). Each diagram is presented on its own page for clarity.

4.3.1 Patient Registration and Email Verification

This flow (Figure 4.2) shows account creation: the backend creates a User record together with the Patient profile, sends a verification code through the email service, and returns a 'verification required' response. After the patient enters the code, the backend verifies it, activates the account, returns a JWT token, and the frontend redirects to the patient portal.

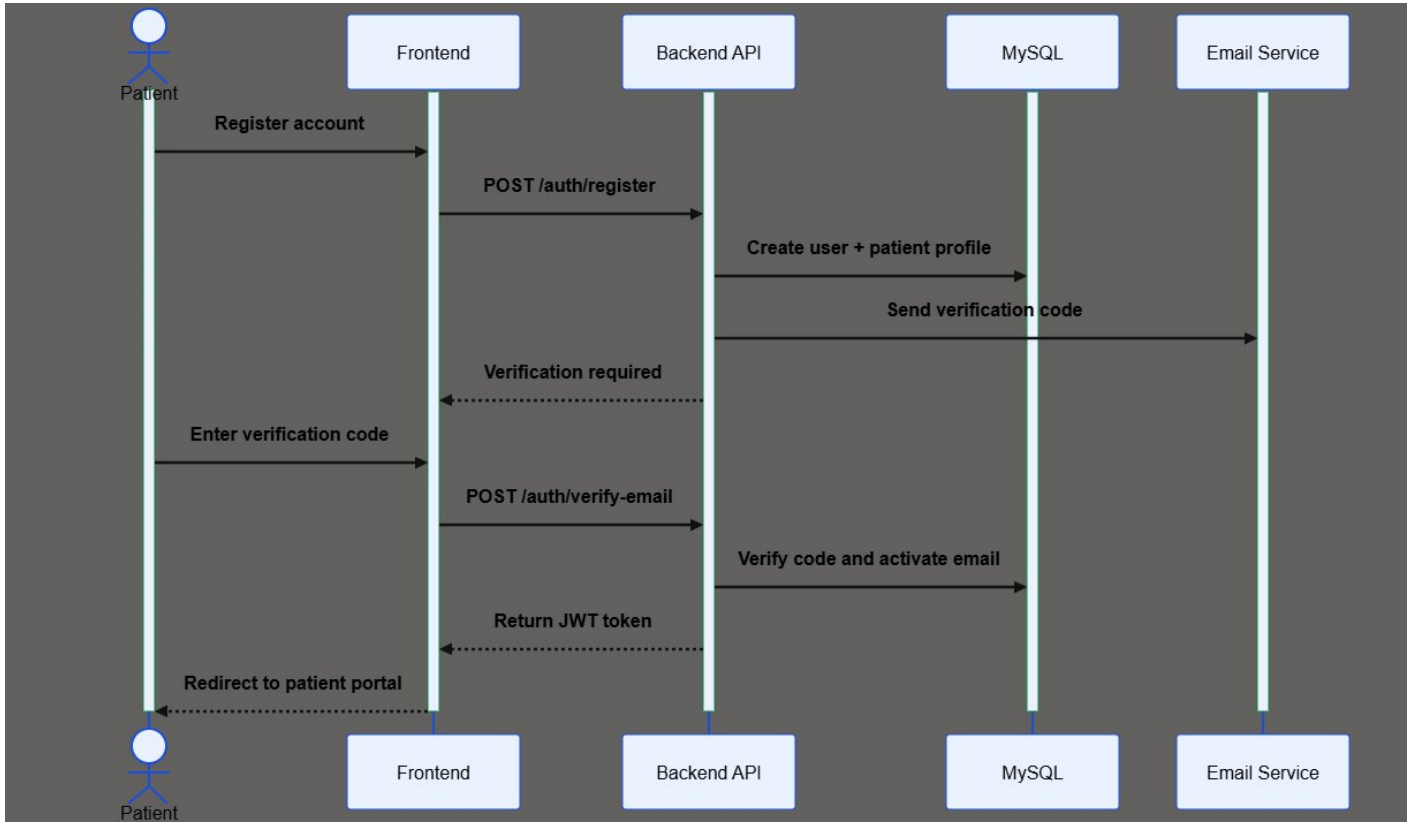

Figure 4.2: Sequence Diagram — Patient Registration and Email Verification

4.3.2 Patient Appointment Booking

This flow (Figure 4.3) shows appointment booking. After the patient selects a category, doctor, and date, the frontend requests available slots; the backend checks booked appointments and returns the free slots. When the patient submits a chosen time, the backend re-checks slot availability. An alt fragment then handles two outcomes: if the slot is available, the appointment is created with a pending status (booking success); if the slot was already booked, a booking error is returned.

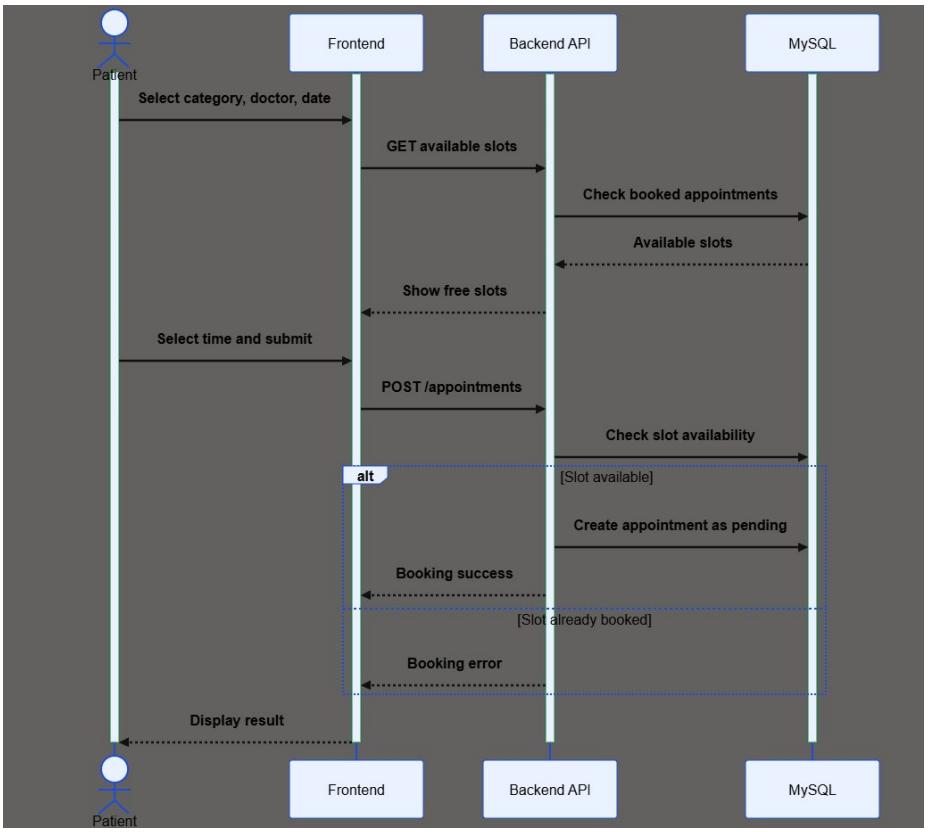

Figure 4.3: Sequence Diagram — Patient Appointment Booking

4.3.3 Order Checkout — Cash on Delivery vs Stripe

This flow (Figure 4.4) shows product checkout. The cart is previewed and validated against current prices and stock. After the patient selects a payment method, an alt fragment splits into two branches: the Cash on Delivery branch creates the order and decreases stock immediately; the Stripe branch creates an awaiting-payment order, opens a Stripe checkout session, and only marks the order paid and decreases stock after the payment webhook is received.

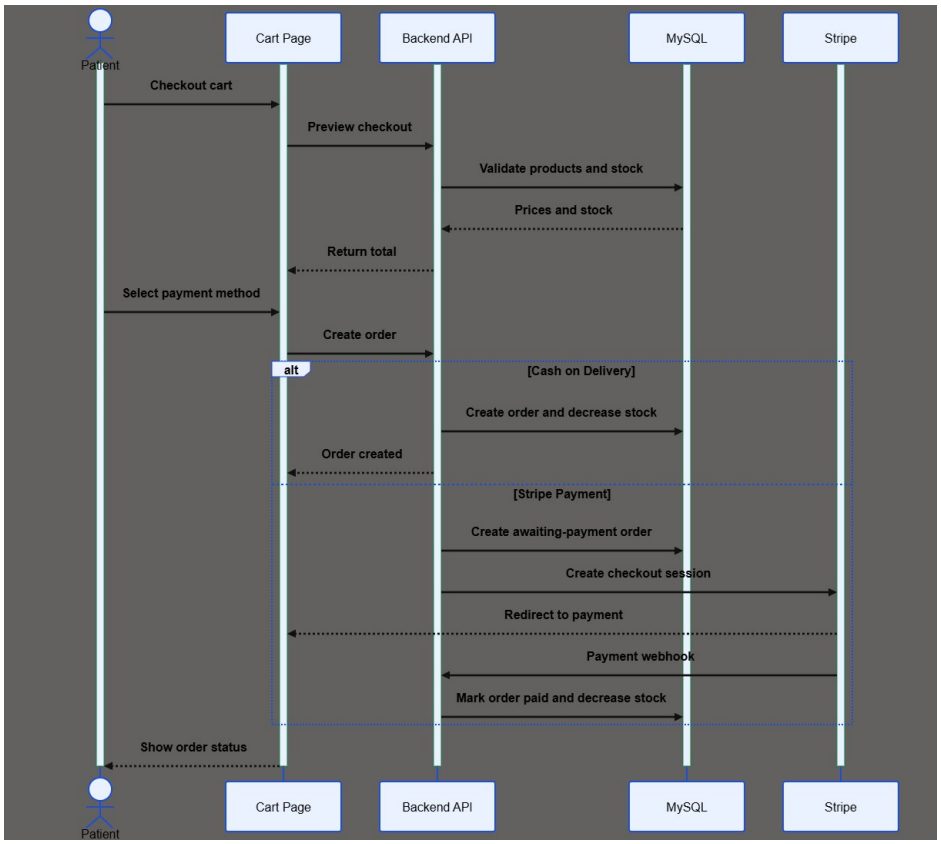

Figure 4.4: Sequence Diagram — Order Checkout (COD vs Stripe)

4.3.4 Review Submission and Admin Moderation

This flow (Figure 4.5) shows the review lifecycle. When a patient submits a review, the backend checks the appointment rules; an alt fragment then either saves the review with a pending status (valid review) or returns an error (invalid review). In the second part, the admin opens the moderation panel, loads the pending reviews, and approves a review — updating its status to approved.

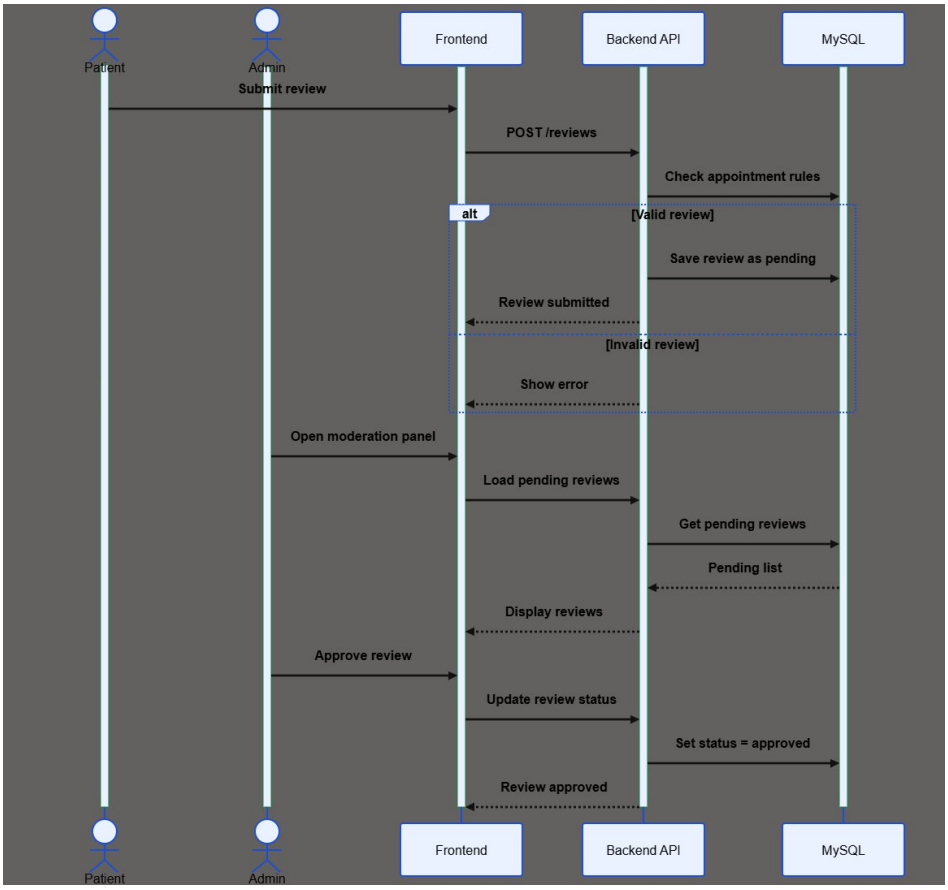

Figure 4.5: Sequence Diagram — Review Submission and Admin Moderation

4.4 Entity Relationship Diagram

The entity relationship diagram (Figure 4.6, next page) documents the relational schema of the pureskin_clinic MySQL database, comprising 14 tables managed through 19 versioned migration files. Each table shows its columns with data types and the key constraints: Primary Key (PK), Unique Key (UK), and Foreign Key (FK). The relationship lines use crow's-foot notation to indicate cardinality.

The following table summarizes all foreign key relationships and their delete behavior:

From Table	Column	References	On Delete
Patients	user_id	Users.id	CASCADE
MedicalRecords	patient_id	Patients.id	CASCADE
Doctors	user_id	Users.id	CASCADE
Appointments	patient_id	Patients.id	CASCADE
Appointments	doctor_id	Doctors.id	CASCADE
TreatmentSessions	appointment_id	Appointments.id	CASCADE
Messages	sender_id / receiver_id	Users.id	CASCADE
Orders	patient_id	Patients.id	CASCADE
OrderItems	order_id	Orders.id	CASCADE
OrderItems	product_id	Products.id	RESTRICT
Reviews	patient_id	Patients.id	CASCADE
Reviews	appointment_id	Appointments.id	CASCADE
ProductStockAdjustments	product_id	Products.id	CASCADE
ProductStockAdjustments	created_by_user_id	Users.id	RESTRICT

Two columns carry UNIQUE constraints that enforce business rules at the database level: Reviews.appointment_id (one review per appointment) and Orders.stripe_checkout_session_id (no duplicate Stripe session linkage). The Migrations table tracks which schema migrations have been applied, supporting controlled and repeatable database evolution.

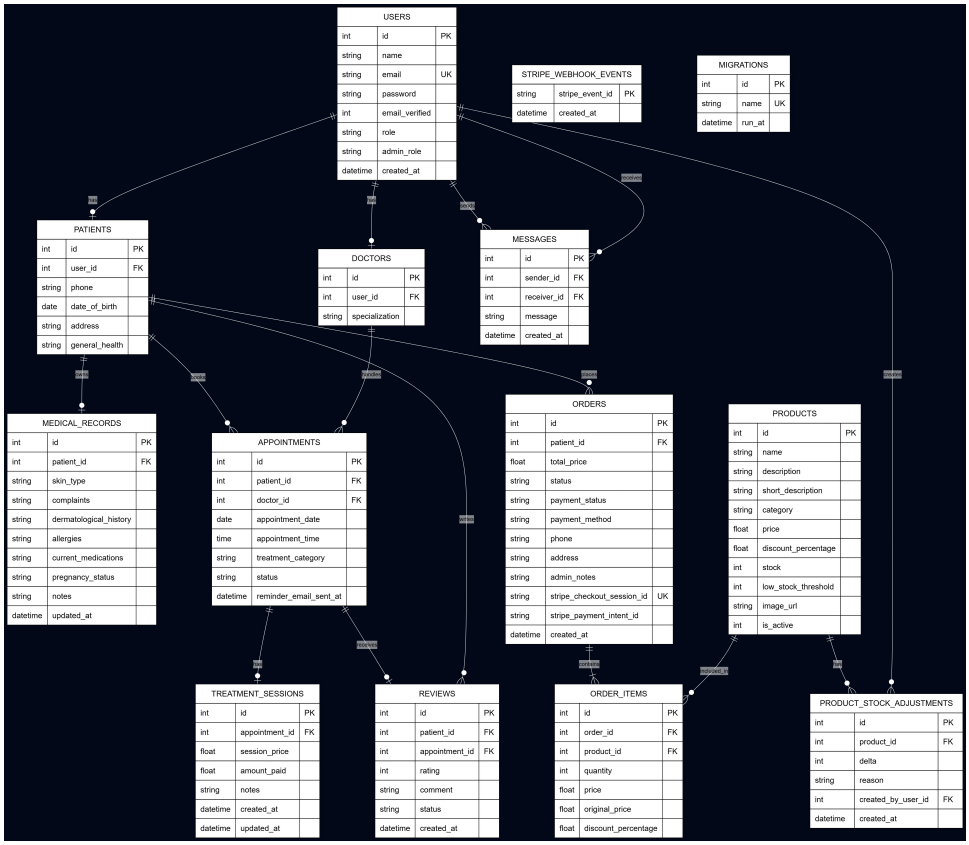

Figure 4.6: PureSkin Clinic — Entity Relationship Diagram (14 Tables)

4.5 Activity Diagrams

The following two activity diagrams document the system's most important and validation-intensive process flows. Each diagram is presented in full on its own page, flowing from top to bottom.

4.5.1 Patient Appointment Booking

This process (Figure 4.7) incorporates three validation gates, each with its own failure path: a slot-availability check after the patient selects a doctor and date, a field-validation check on submission, and a final concurrency check that re-confirms the slot is still free before creating the appointment. Only when all three gates pass is the appointment created with a pending status, an optional email sent, and a success message shown — otherwise the corresponding error is displayed.

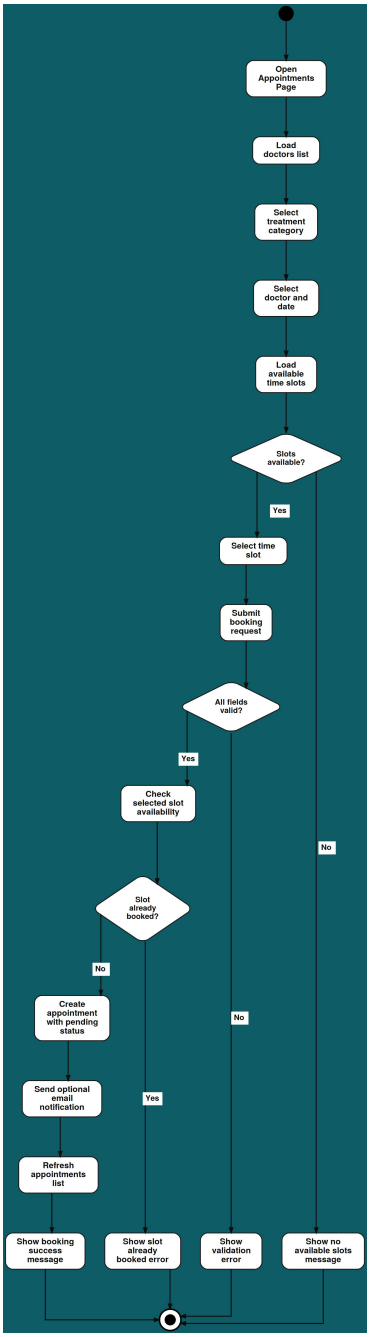

Figure 4.7: Activity Diagram — Patient Appointment Booking

4.5.2 Order Checkout

This process (Figure 4.8) shows the full checkout flow with its decision points. After the cart is opened and contact details are entered, the system validates that phone and address are provided, then branches on the payment method. The Cash on Delivery branch validates stock, decreases it, creates the order, and sends a confirmation email. The Stripe branch creates an awaiting-payment order, opens a checkout session, and redirects to Stripe; upon receiving the payment webhook, an idempotency check prevents duplicate processing before the order is locked, stock decreased, and the order marked paid.

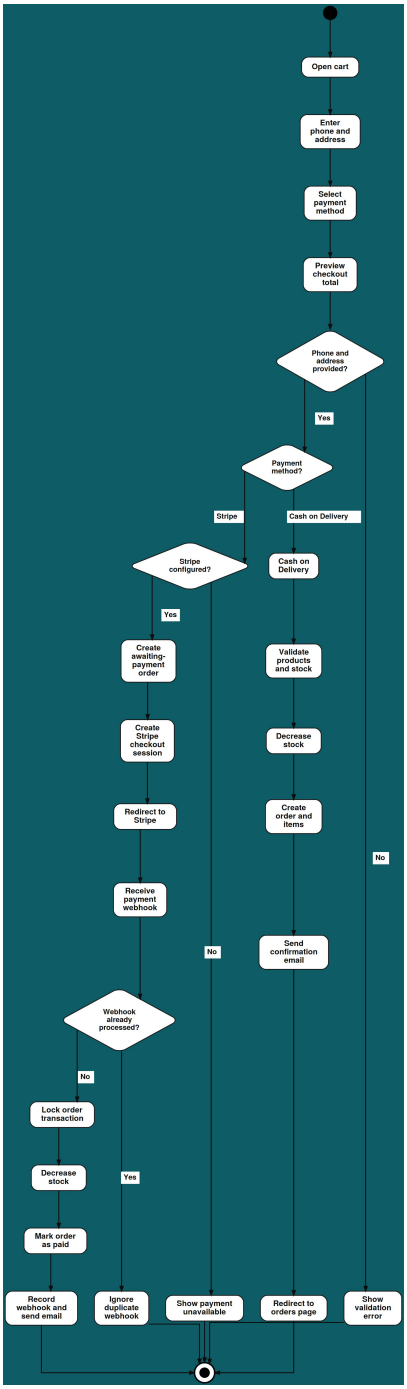

Figure 4.8: Activity Diagram — Order Checkout (COD vs Stripe)

4.6 Conclusion and Recommendations

This chapter presented the complete system design of PureSkin Clinic through eight design artifacts: a class diagram covering 13 domain entities, four sequence diagrams tracing the principal interaction flows (registration, booking, checkout, and review moderation), a 14-table entity relationship diagram, and two activity diagrams documenting the booking and checkout processes. All artifacts are derived from the implemented codebase and are consistent with the functional requirements of Chapter Three.

The design reflects several deliberate engineering decisions: stateless JWT authentication supporting horizontal scalability, database-level constraints enforcing business rules (unique reviews, guarded stock updates), idempotent webhook processing preventing duplicate order fulfillment, and a versioned migration system enabling controlled schema evolution.

It is recommended that future design iterations address the following:

- Introduce a formal API versioning strategy (e.g., /api/v1/) to support backward-compatible evolution of the REST interface.
- Replace the local filesystem image store with a cloud object storage design for improved scalability and availability.
- Introduce a caching layer (e.g., Redis) for product catalog and appointment slot queries to reduce database load under high traffic.
- Design a WebSocket-based messaging layer to replace the current polling interval in the Messages module.

Chapter Five presents the coding and implementation details, including the development environment setup, programming languages and frameworks used, and screenshots of the deployed system.

Chapter Five: Coding and Implementation

5.1 Introduction

This chapter presents the coding and implementation of the PureSkin Clinic system. It describes the programming languages and technologies used to build the application, the database system and its schema, the development environment in which the system was implemented, and the relationship between the design-stage activity diagrams and their concrete implementation. It then documents the implemented system interface through screenshots taken from the live system, organized by user role and presented in the order of the real system workflow.

The system is a full-stack web application that is fully implemented and deployed in production at https://pureclinic.online, with its REST API served from https://api.pureclinic.online/api. The interface is role-based, serving four actors: Guest, Patient, Doctor, and Administrator.

5.2 Coding Programming Languages

PureSkin Clinic was implemented as a full-stack JavaScript application, separated into a frontend single-page application, a backend REST API, a relational database, and a set of external services. This section describes the implementation of each layer.

5.2.1 Frontend Implementation

The frontend was implemented as a single-page application (SPA) using React with the Vite build tool, written in JavaScript and JSX. Tailwind CSS was used for responsive styling, and Axios was used to communicate with the backend REST API. Client-side routing is handled by React Router, and the interface supports both English and Arabic through an internationalization layer.

5.2.2 Backend Implementation

The backend was implemented as a REST API using Node.js and the Express framework, following a layered architecture in which routes delegate to controllers, controllers call data-access models, and shared logic is placed in service and helper modules. Authentication is implemented using JSON Web Tokens (JWT), while

authorization is enforced by role-based middleware that distinguishes between Patient, Doctor, and Admin, with additional permission checks for administrative sub-roles.

The two code snippets below show the JWT authentication middleware and the role-based authorization middleware that together secure every protected endpoint.

server/middleware/auth.js (JWT authentication)

```
function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization || '';
  const token = authHeader.startsWith('Bearer ')
    ? authHeader.substring(7) : null;
  if (!token) return res.status(401)
    .json({ message: 'Authentication required' });
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.user = { id: payload.id, role: payload.role,
      admin_role: payload.admin_role };
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Invalid or expired token' });
  }
}
```

server/middleware/role.js (Role-based authorization)

```
function roleMiddleware(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user) return res.status(401)
      .json({ message: 'Authentication required' });
    if (!allowedRoles.includes(req.user.role))
      return res.status(403).json({ message: 'Forbidden' });
    next();
  };
}
```

User passwords are never stored in plaintext. During registration, each password is salted and hashed with bcrypt before the account is created.

server/controllers/authController.js (Password hashing)

```
const salt = await bcrypt.genSalt(10);
const passwordHash = await bcrypt.hash(req.body.password, salt);
const user = await createUser({ name, email, passwordHash,
  role: 'patient', emailVerified: false });
```

5.2.3 Database Implementation

MySQL was used for persistent relational storage through the mysql2 driver, which manages a pooled set of connections. The database schema is created and evolved through a sequence of versioned migration files executed automatically when the server starts, ensuring that every environment converges to the same schema in a controlled order.

server/config/db.js (Migration runner)

```
async function runMigrations(pool) {
  await pool.query(`CREATE TABLE IF NOT EXISTS Migrations (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL UNIQUE,
    run_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
  const [rows] = await pool.query('SELECT name FROM Migrations');
  const applied = new Set(rows.map(r => r.name));
  for (const file of migrationFiles) {
    if (applied.has(file)) continue;
    await require(file).up(pool);
    await pool.query('INSERT INTO Migrations (name) VALUES (?)', [file]);
  }
}
```

5.2.4 External Services

Two external services are integrated into the system. Nodemailer, configured with an SMTP provider, is used to send transactional email notifications such as the registration verification code and appointment and order confirmations. Stripe is used to process online card payments through Stripe Checkout, alongside a Cash on Delivery option. Because Stripe may deliver the same webhook event more than once, the system records every processed event identifier and ignores duplicates, ensuring that each order is fulfilled exactly once. Stock is also decremented with a guarded update that cannot oversell a product.

server/controllers/stripeWebhookController.js (Webhook idempotency)

```
async function fulfillCheckoutSession(session, stripeEventId) {
  if (await webhookEventAlreadyProcessed(stripeEventId)) {
    return; // already handled: idempotent no-op
  }
  // ... fulfil order in a transaction ...
}
```

server/models/productModel.js (Stock decrement guard)

```
async function decrementStockGuarded(connection, productId, quantity) {
  const [result] = await connection.query(
    'UPDATE Products SET stock = stock - ? WHERE id = ? AND stock >= ?',
    [quantity, productId, quantity]);
  return result.affectedRows; // 0 means insufficient stock
}
```

5.3 Database System

The system uses a MySQL relational database named pureskin_clinic. The schema is composed of fourteen tables that store authentication data, clinical data, e-commerce data, and supporting audit and control data. Relationships between entities are preserved using foreign keys, and all schema changes are applied through versioned migrations; a dedicated Migrations table records which migrations have already been applied so that each one runs exactly once. The tables and their purposes are summarized below.

Table Name	Purpose
Users	Stores authentication credentials and role information.
Patients	Stores patient profile information.
Doctors	Stores doctor profile and specialization.
Appointments	Stores appointment bookings and their status.
MedicalRecords	Stores patient medical-profile data.
Products	Stores clinic store products.
Orders	Stores patient orders.
OrderItems	Stores the products contained in each order.
Reviews	Stores patient feedback and its moderation status.
Messages	Stores messages exchanged between users.

Table Name	Purpose
TreatmentSessions	Stores treatment and billing session details.
ProductStockAdjustments	Stores inventory audit records.
StripeWebhookEvents	Prevents duplicate Stripe webhook processing.
Migrations	Tracks which database schema updates have been applied.

5.4 Establishment of the Development Environment

The system was implemented using a modern development environment that combines a code editor, version control, and separate local servers for the frontend and backend. The Cursor editor (a Visual Studio Code–based IDE) was used to write and debug both the React frontend and the Node.js backend. The project is divided into four main parts: a client folder containing the React application, a server folder containing the Express API, a database folder containing the SQL schema, and a documentation folder. Figure 5.1 shows the project structure open in the development environment.

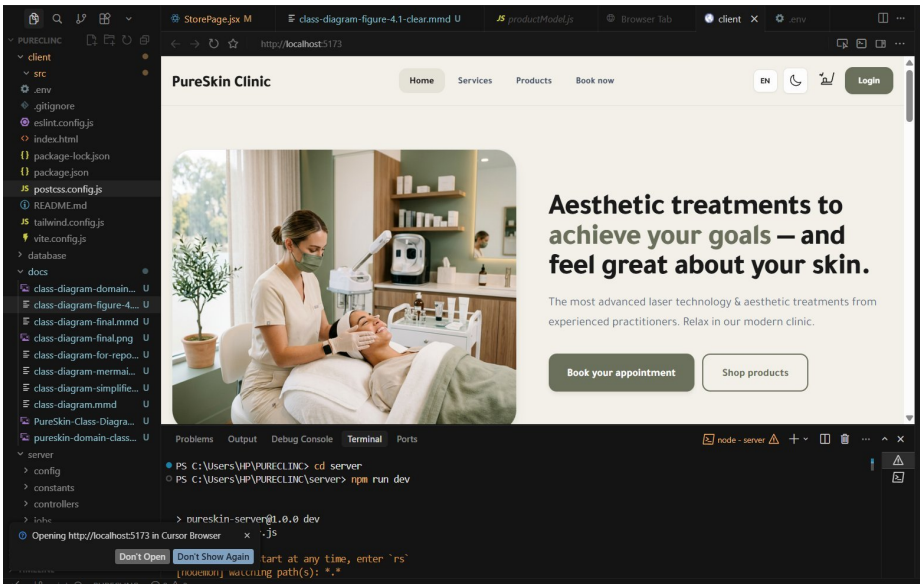

Figure 5.1: PureSkin Clinic project structure in the development environment (client, server, database, and documentation).

Source code was managed using Git, with the repository hosted on GitHub. Version control was used both to track changes during development and to support the automated deployment workflow, which is triggered when changes are pushed to the

main branch. Figure 5.2 shows the repository and its commit history managed through GitHub Desktop, including a change to the deployment workflow file.

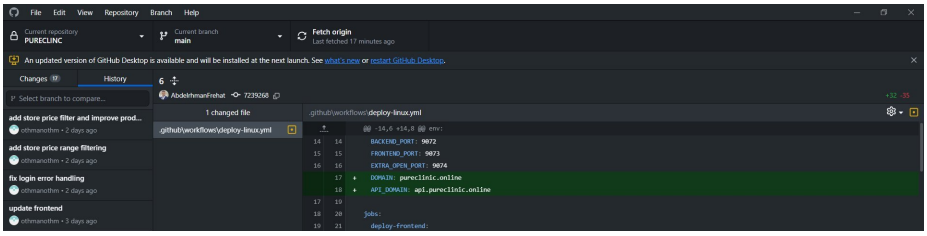

Figure 5.2: Version control and repository management using GitHub Desktop.

During local development, the two applications run side by side. The frontend is started with the command npm run dev, which launches the Vite development server with hot-module reloading. The backend is run with Node.js using nodemon, which automatically restarts the server when source files change. The frontend communicates with the backend through a configurable API base URL, allowing the same code to run against a local server during development and against the production API after deployment.

5.5 Activity Diagrams

The main activity diagrams of the system were presented in Chapter Four as part of the system design. They are not repeated here; instead, this section explains how those design-stage processes were realized in the implemented code, linking each process to the interfaces and backend modules that implement it.

The appointment booking flow was implemented in the patient booking interface together with the appointment routes, controller, and model: the frontend loads available slots, and the backend validates the slot and creates the appointment with a pending status. The checkout flow was implemented in the shopping cart interface together with the order controller and the Stripe webhook controller: the Cash on Delivery branch creates the order and decreases stock immediately, while the Stripe branch defers stock decrement until the payment webhook is received and verified. The review moderation flow was implemented through the patient review interface and the administrator users interface, backed by the review model: a submitted review is stored with a pending status and only becomes publicly visible after an administrator approves it. The screenshots in the following section show the interfaces through which these implemented processes are operated.

5.6 System Interface (Input / Output Design)

This section presents the implemented user interface of PureSkin Clinic through screenshots captured from the live system. The interfaces are organized by actor and presented in the order of the real system workflow, beginning with the public and authentication interfaces and continuing through the patient, doctor, and administrator interfaces. For each interface, the input data, the processing performed by the backend, and the resulting output are described.

Note: some screenshots contain sample personal data (names, email addresses, and phone numbers) used during testing.

5.6.1 Public and Authentication Interfaces

The public landing page is the entry point of the system. It presents the clinic's services and provides links to book an appointment or shop for products, as well as access to the login and registration pages. Figure 5.3 shows the home page.

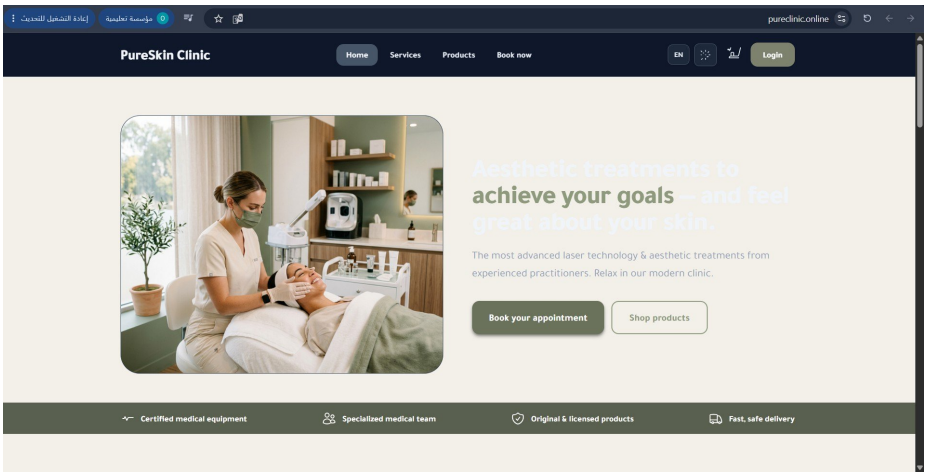

Figure 5.3: PureSkin Clinic public home (landing) page.

New and returning users access the system through the registration and login interface shown in Figure 5.4.

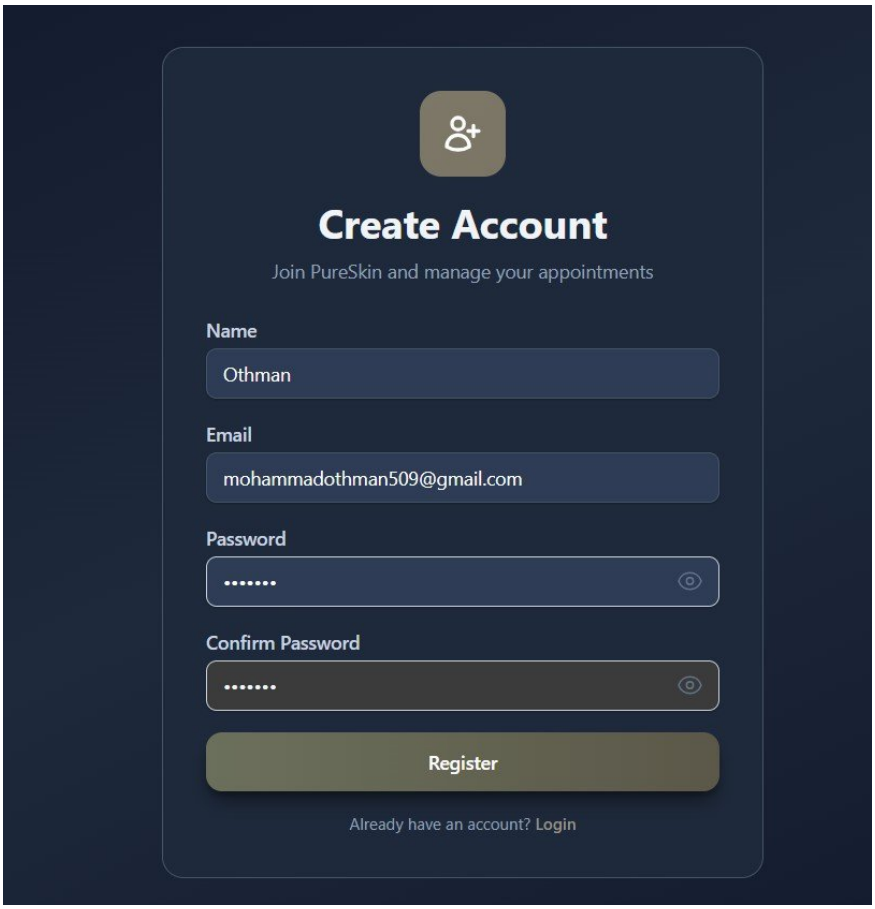

Figure 5.4: Patient registration and login interface.

Login / Registration interface

Input: email and password for login, or name, email, and password for registration. **Processing:** the backend validates the submitted data, verifies the password against its stored bcrypt hash, and generates a signed JWT token. **Output:** the user is authenticated and redirected to the dashboard that corresponds to their role (patient, doctor, or admin).

Registration is protected by email verification. After a patient registers, the system emails a six-digit verification code, as shown in Figure 5.5.

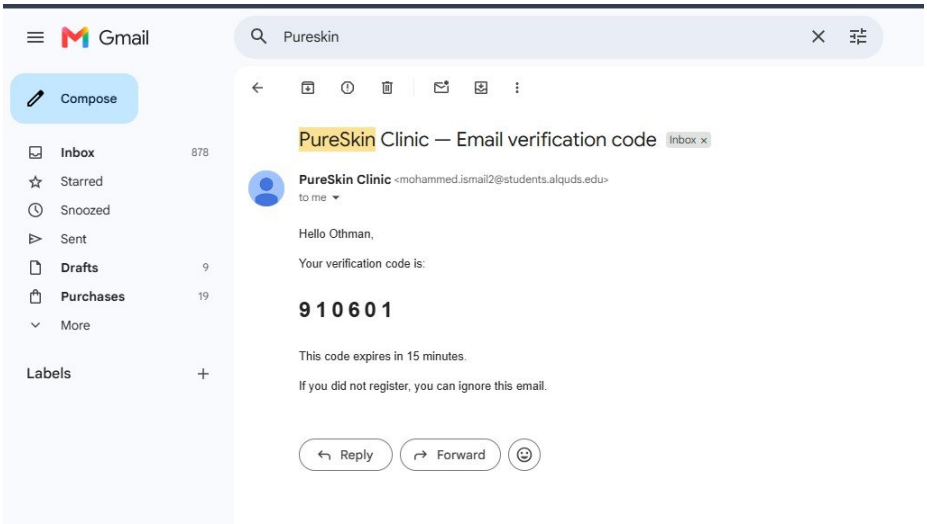

Figure 5.5: Email verification code sent to the patient's email.

Email verification

Input: the six-digit verification code sent to the patient's email. **Processing:** the backend compares the entered code against the stored hashed code and, if valid, marks the account as verified. **Output:** the patient's account is activated and the patient can sign in and use the system.

5.6.2 Patient Interfaces

After signing in, the patient accesses a dedicated portal that supports appointment booking, medical-profile management, messaging, shopping, ordering, and feedback. The interfaces are presented below in workflow order.

The patient dashboard, shown in Figure 5.6, summarizes the patient's appointments, orders, and total spending, and provides quick links to all patient features.

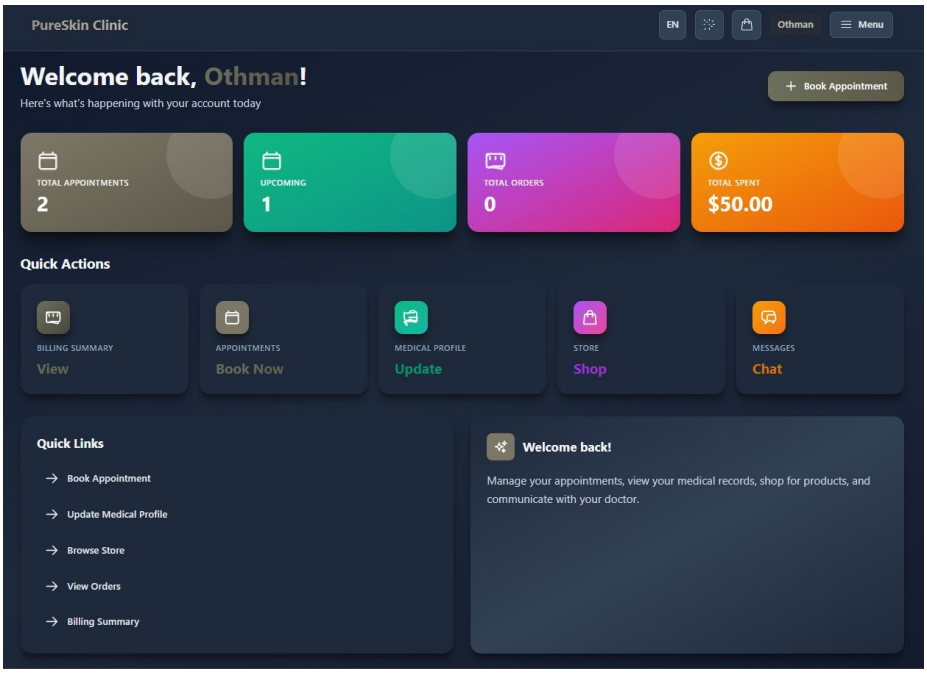

Figure 5.6: Patient dashboard after successful login.

From the booking interface in Figure 5.7, the patient schedules a consultation with a doctor.

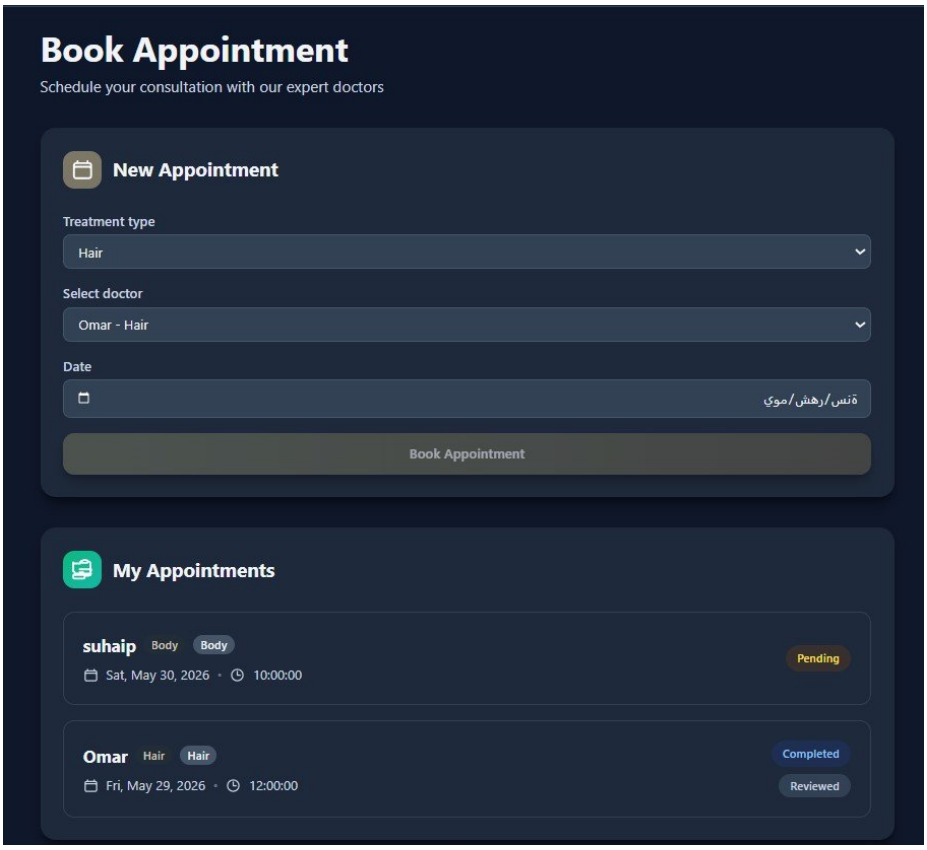

Figure 5.7: Patient appointment booking interface.

Book Appointment interface

Input: treatment category, doctor, and date (and a time slot once slots are loaded). **Processing:** the backend checks the doctor's booked appointments, returns the available slots, and creates a new appointment with a pending status. **Output:** the appointment request is submitted and listed under the patient's appointments.

Immediately after booking, the system sends an email confirming that the appointment request was received and is pending confirmation, as shown in Figure 5.8.

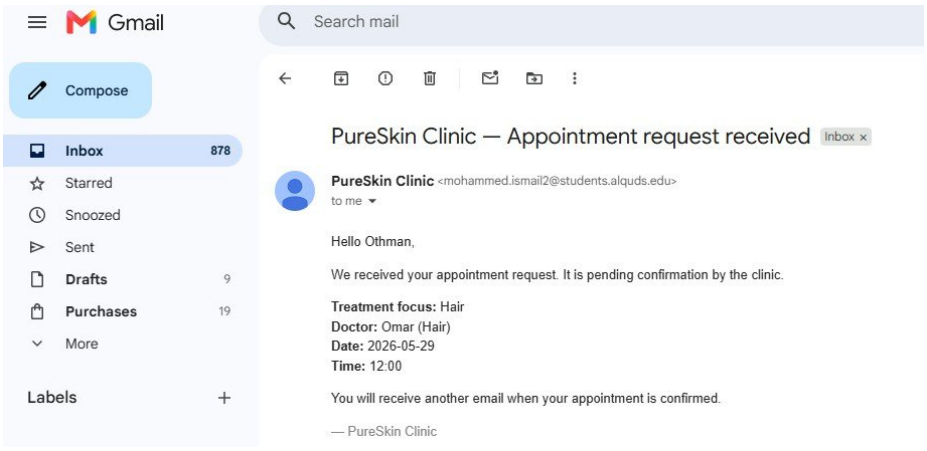

Figure 5.8: Appointment request received email notification.

When a doctor confirms the appointment, a second email is sent to the patient, shown in Figure 5.9.

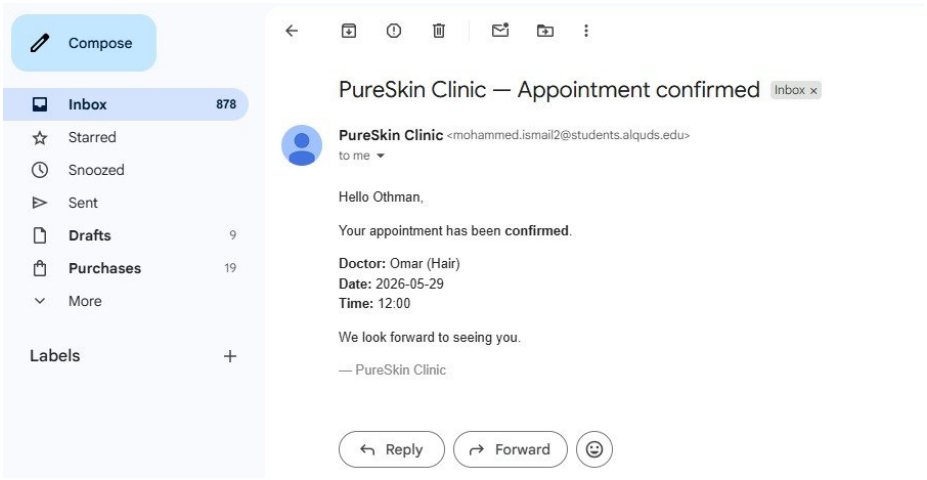

Figure 5.9: Appointment confirmation email notification.

The medical profile interface in Figure 5.10 lets the patient maintain the clinical information used by the treating doctor.

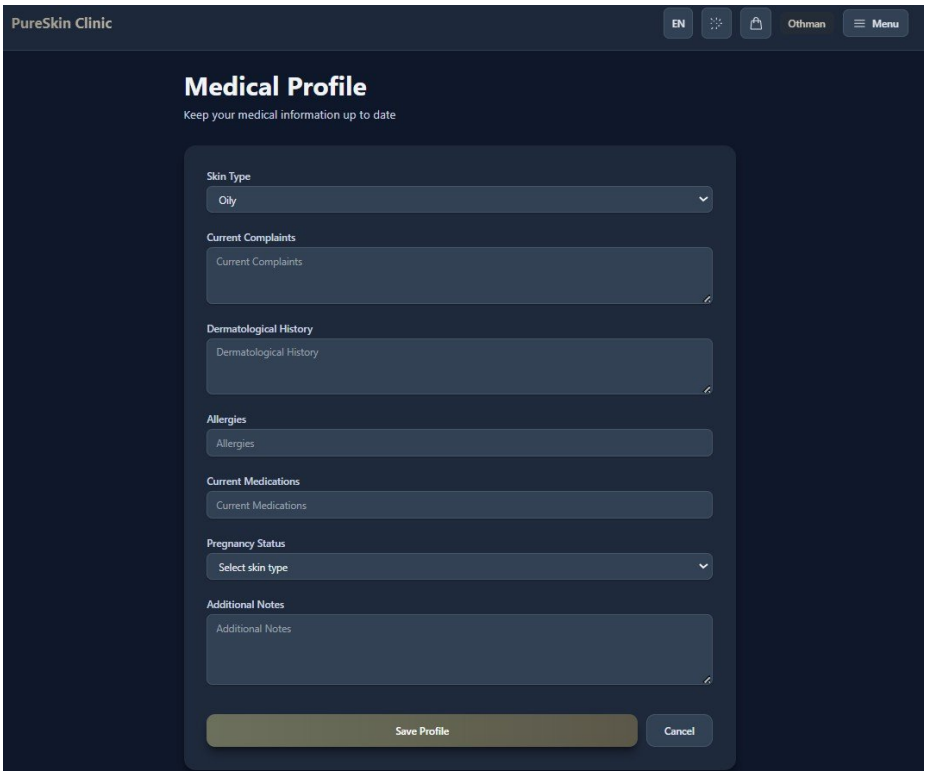

Figure 5.10: Patient medical profile interface.

Medical Profile interface

Input: skin type, current complaints, dermatological history, allergies, current medications, pregnancy status, and additional notes. **Processing:** the backend validates the data and creates or updates the patient's medical record. **Output:** the patient's profile and medical record are updated and stored.

The messaging interface in Figure 5.11 enables direct communication between the patient and their doctor.

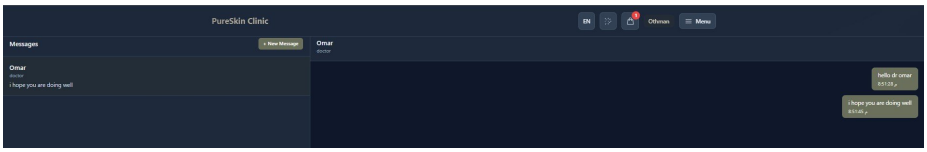

Figure 5.11: Patient messaging interface.

Messages interface

Input: the message text and the selected receiver (doctor). **Processing:** the backend stores the message and associates it with the conversation between the two users. **Output:** the stored message appears in the conversation thread, which refreshes periodically.

The store interface in Figure 5.12 presents the clinic's products for purchase.

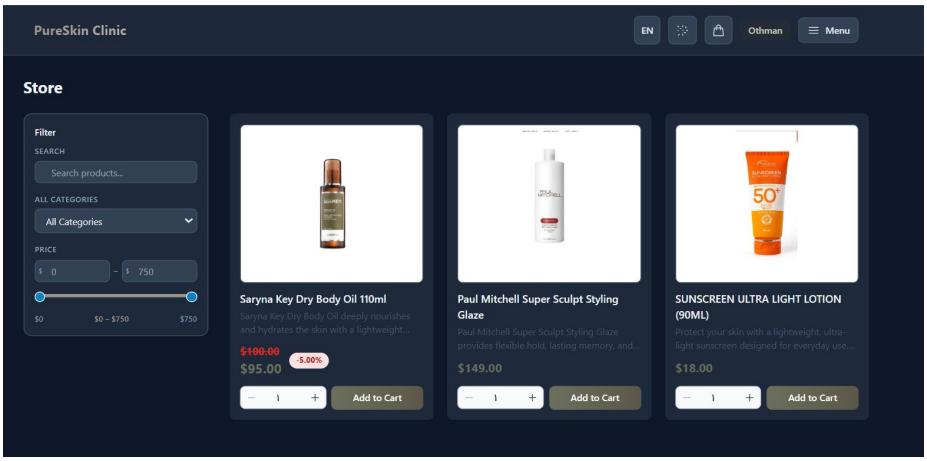

Figure 5.12: Patient product store interface.

Store interface

Input: search text, category selection, and price-range filters. **Processing:** the backend (and client-side filter) returns the products matching the selected criteria. **Output:** a filtered list of products is displayed, each with its price and any discount.

The shopping cart and checkout interface in Figure 5.13 completes a purchase.

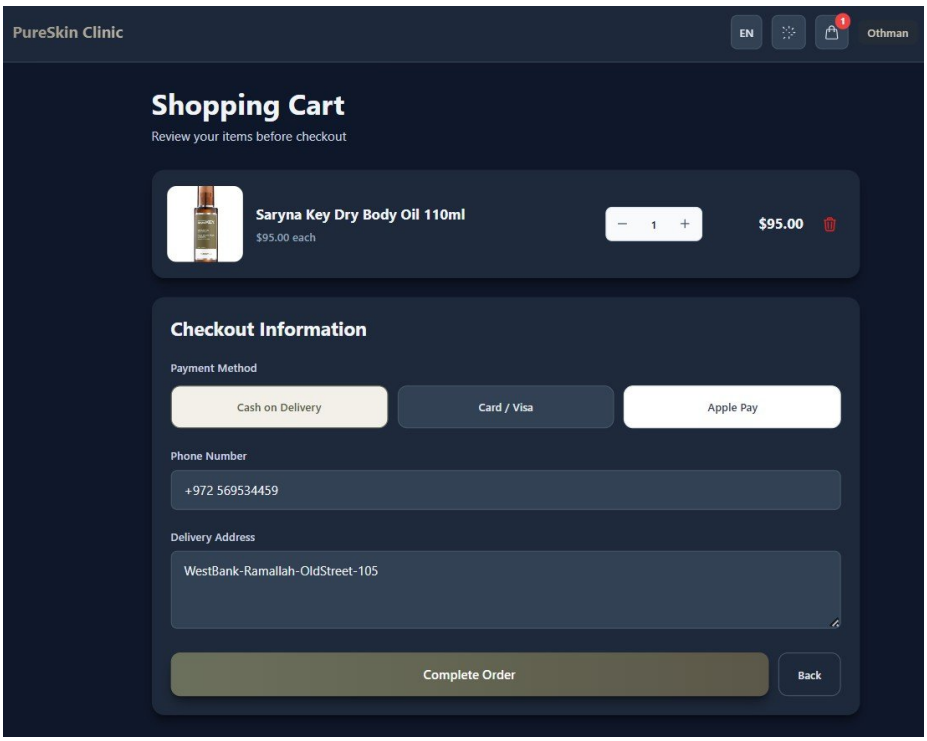

Figure 5.13: Shopping cart and checkout interface.

Shopping Cart interface

Input: selected products and quantities, phone number, delivery address, and payment method (Cash on Delivery, Card, or Apple Pay). **Processing:** a checkout preview validates

current prices and stock; for Cash on Delivery the order is created and stock decreased, while for card payment a Stripe Checkout session is created. **Output:** an order is created (Cash on Delivery) or the patient is redirected to Stripe to complete the payment.

After the order is placed or paid, the patient receives an order confirmation email, shown in Figure 5.14.

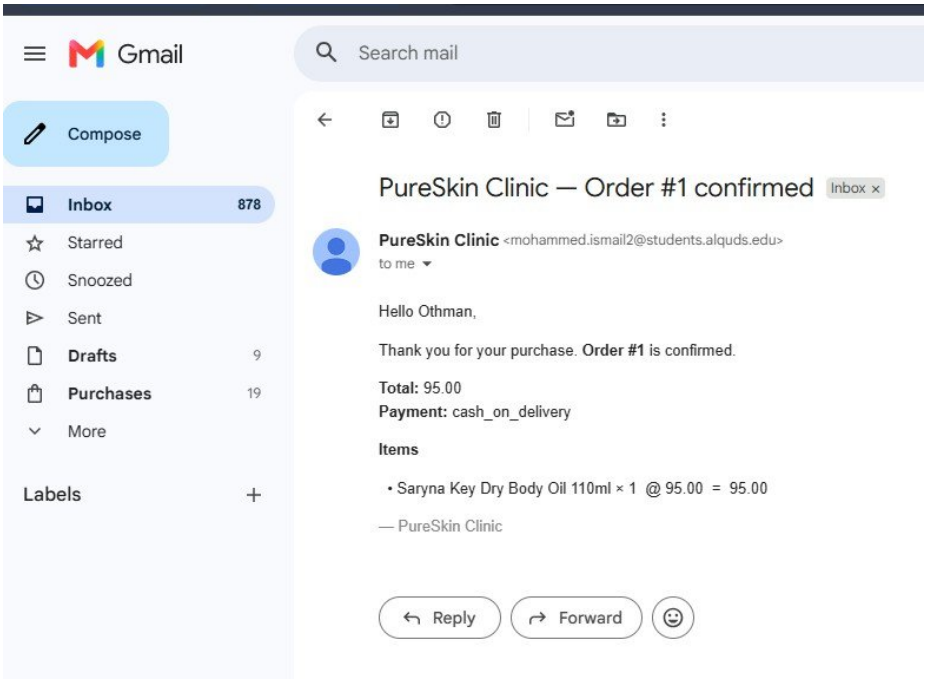

Figure 5.14: Order confirmation email.

The My Orders interface in Figure 5.15 lets the patient track their order history and status.

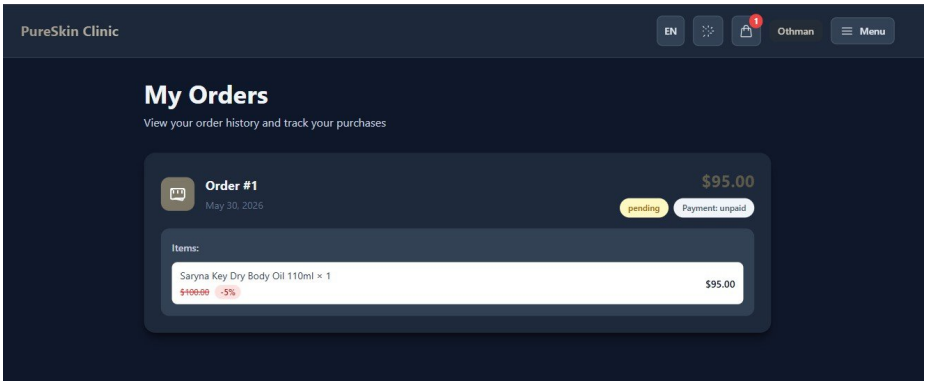

Figure 5.15: Patient order history interface.

After a completed appointment, the patient can submit feedback. Once approved by an administrator, the review is displayed publicly, as shown in Figure 5.16.

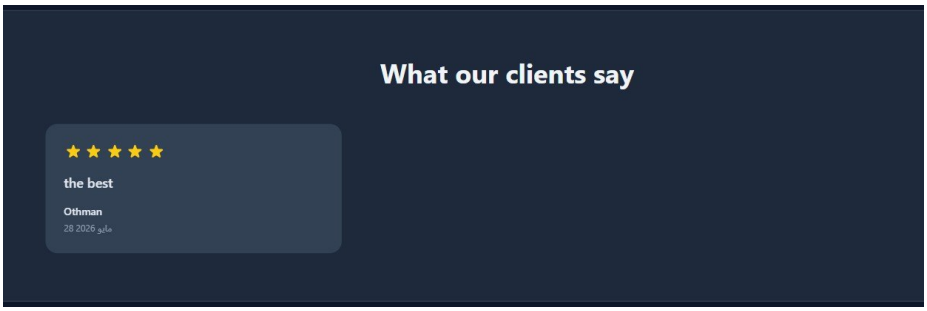

Figure 5.16: Patient review displayed publicly after moderation.

Review / Feedback

Input: a star rating and a written comment for a completed appointment. **Processing:** the backend verifies that the appointment belongs to the patient, is completed, and has not already been reviewed, then stores the review with a pending status until an administrator approves it. **Output:** the approved review appears publicly in the 'What our clients say' section of the landing page.

5.6.3 Doctor Interfaces

The doctor portal allows a doctor to manage their appointment queue, access patient records, and document treatment sessions. The interfaces are presented below in workflow order.

The doctor dashboard in Figure 5.17 summarizes the doctor's appointments and patients.

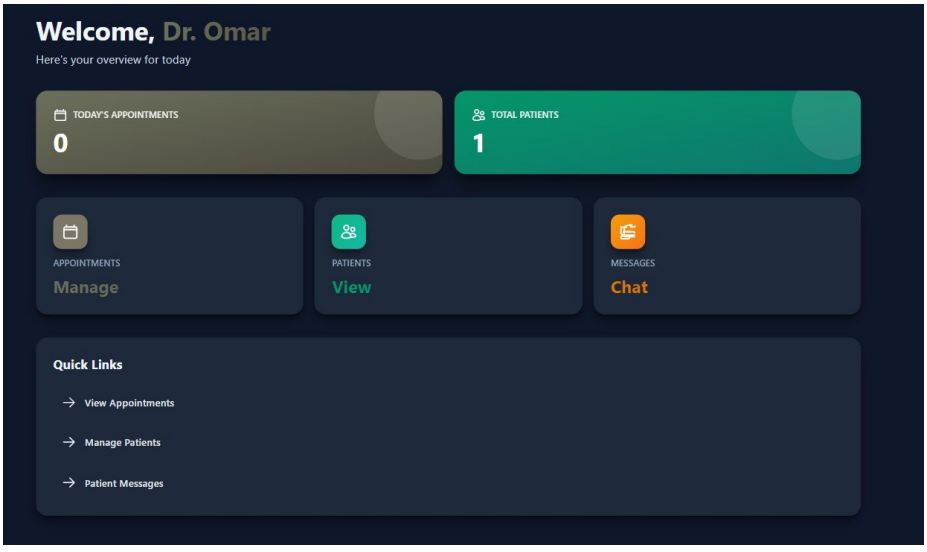

Figure 5.17: Doctor dashboard interface.

The appointment management interface in Figure 5.18 lets the doctor advance each appointment through its lifecycle.

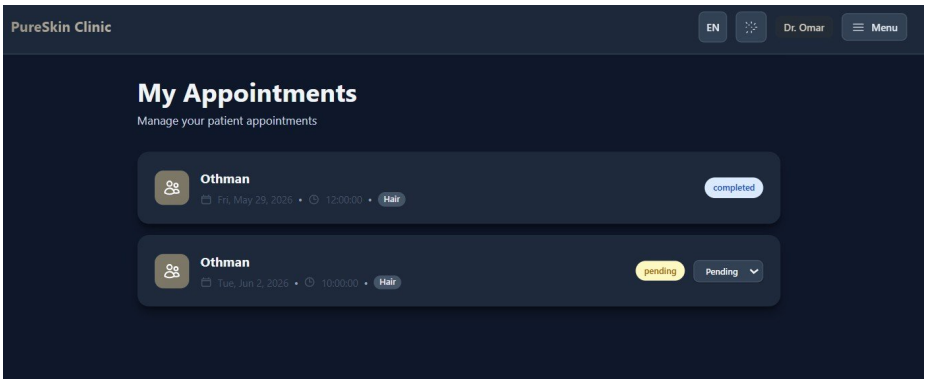

Figure 5.18: Doctor appointment management interface.

Doctor Appointments interface

Input: an appointment status update (confirm, complete, or cancel). **Processing:** the backend verifies that the appointment belongs to the logged-in doctor and that the requested status transition is valid, including the treatment-evidence check required before completion. **Output:** the appointment status becomes confirmed, completed, or cancelled accordingly.

The patient list in Figure 5.19 gives the doctor access to the records of their assigned patients.

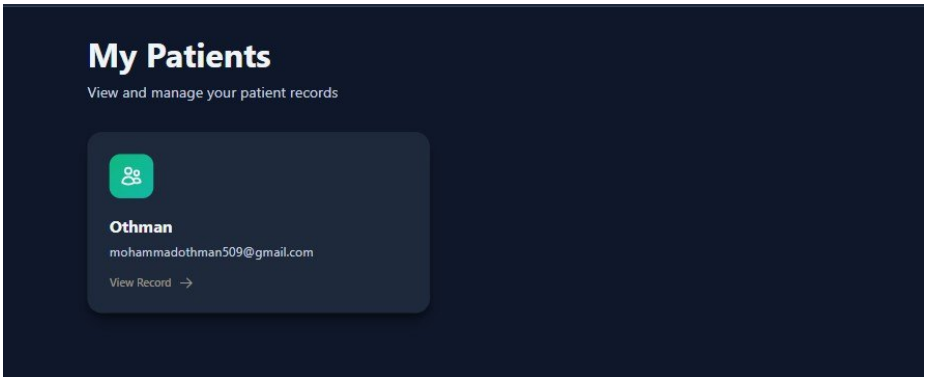

Figure 5.19: Doctor patient list interface.

For each patient, the doctor reviews the full history and records treatment sessions through the interface in Figure 5.20.

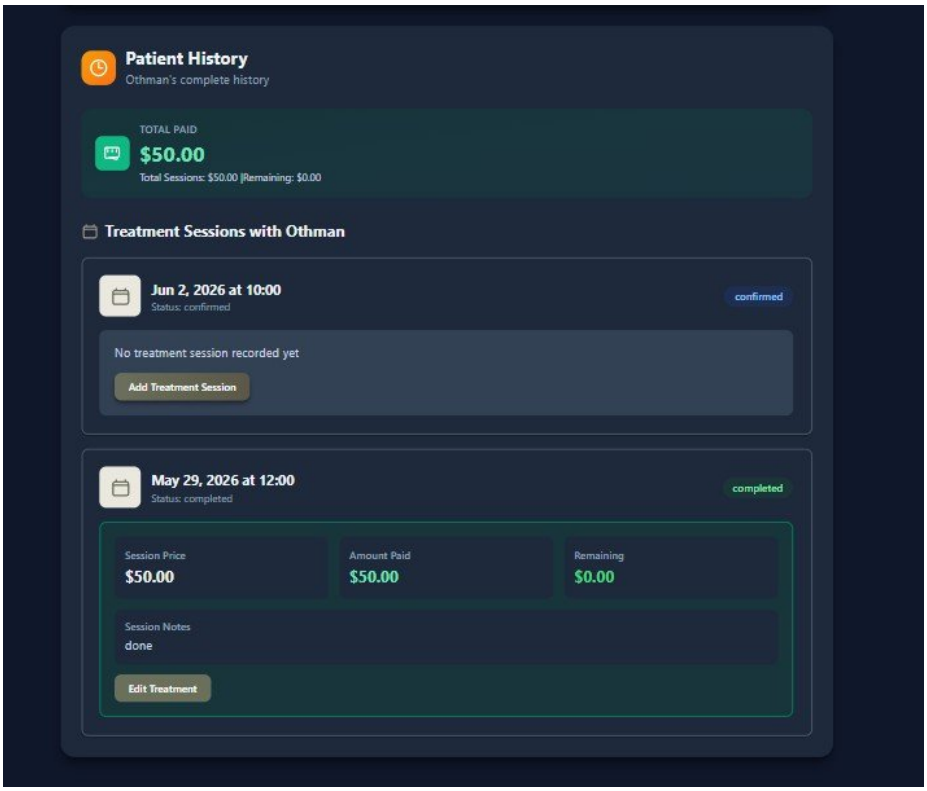

Figure 5.20: Doctor patient history and treatment session interface.

Patient History / Treatment Sessions interface

Input: session price, amount paid, clinical notes, and treatment details. **Processing:** the system stores the treatment session linked to the corresponding appointment and uses it as the evidence required to complete the appointment. **Output:** the patient's treatment history and billing information are updated.

5.6.4 Administrator Interfaces

The administrator portal provides clinic-wide management of users, doctors, products, orders, and financial reporting, with actions gated by granular admin permissions. The interfaces are presented below in workflow order.

The admin dashboard in Figure 5.21 summarizes system-wide statistics.

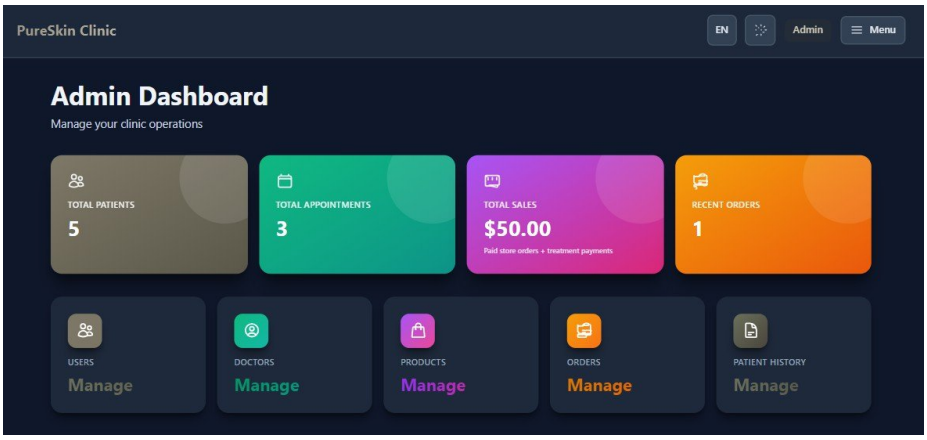

Figure 5.21: Admin dashboard interface.

The dashboard summarizes the total number of patients, appointments, and orders, together with total sales, and provides management entry points for users, doctors, products, orders, and patient history.

The users management interface in Figure 5.22 manages all system accounts.

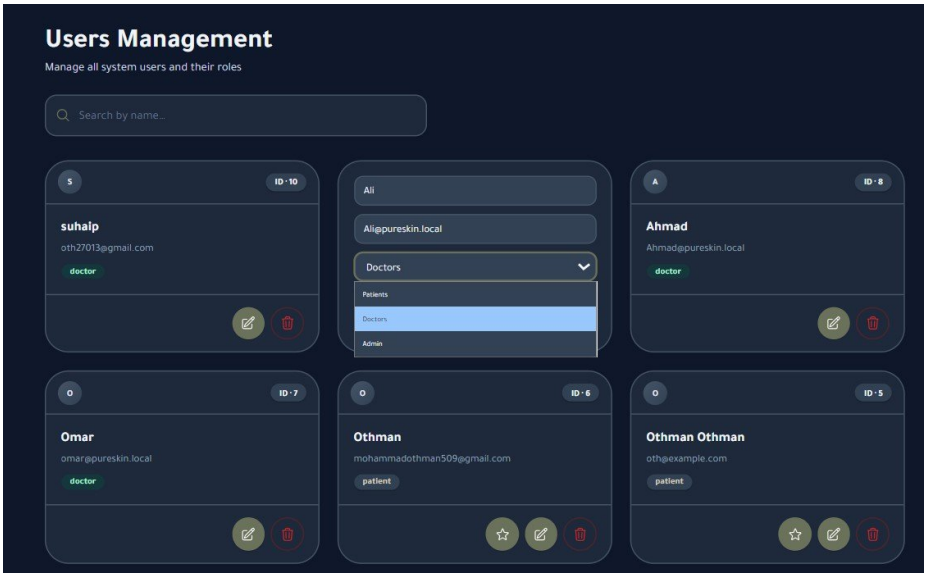

Figure 5.22: Users management interface.

Users Management interface

Input: user search terms and account updates, including role changes. **Processing:** the backend applies the updates and enforces the selected role. **Output:** the updated user information and role assignments are saved and displayed.

The doctors management interface in Figure 5.23 manages doctor accounts.

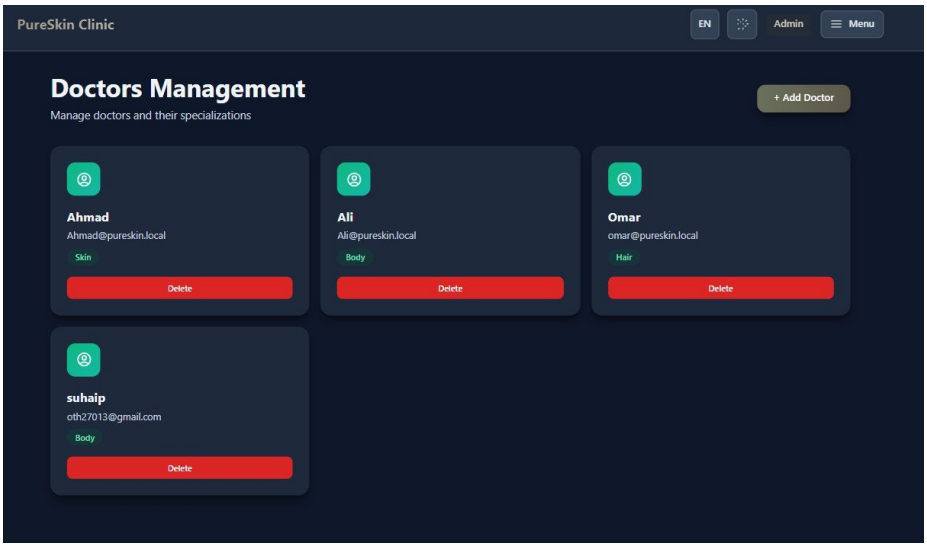

Figure 5.23: Doctors management interface.

Doctors Management interface

Input: doctor name, email, specialization, and password. **Processing:** the backend creates or updates the doctor account and its associated user record. **Output:** the doctor account is created or updated and appears in the list.

The products management interface in Figure 5.24 manages the store catalog and inventory.

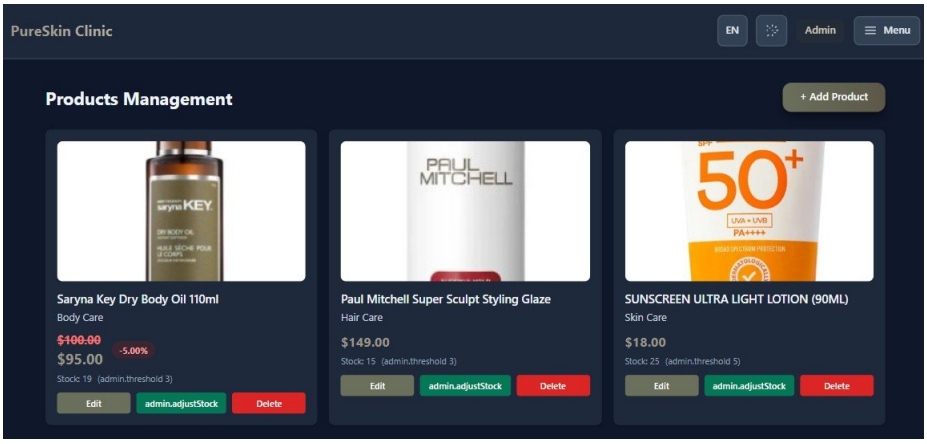

Figure 5.24: Products management and inventory interface.

Products Management interface

Input: product name, category, price, stock, image, and discount. **Processing:** the backend validates the data, stores the product (and uploaded image), and records any stock adjustment in the audit table. **Output:** the product catalog and stock levels are updated.

The orders management interface in Figure 5.25 manages all customer orders.

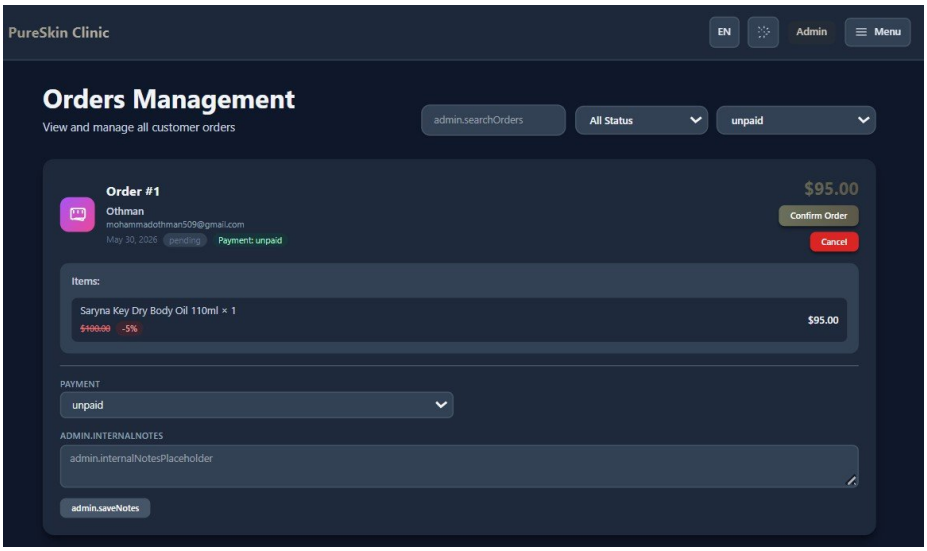

Figure 5.25: Orders management interface.

Orders Management interface

Input: order status, payment status, and internal administrative notes. **Processing:** the backend applies the status and payment updates and records the notes. **Output:** the order status is updated and the related inventory is managed.

The administrator can also review each patient's complete financial history through the interface in Figure 5.26.

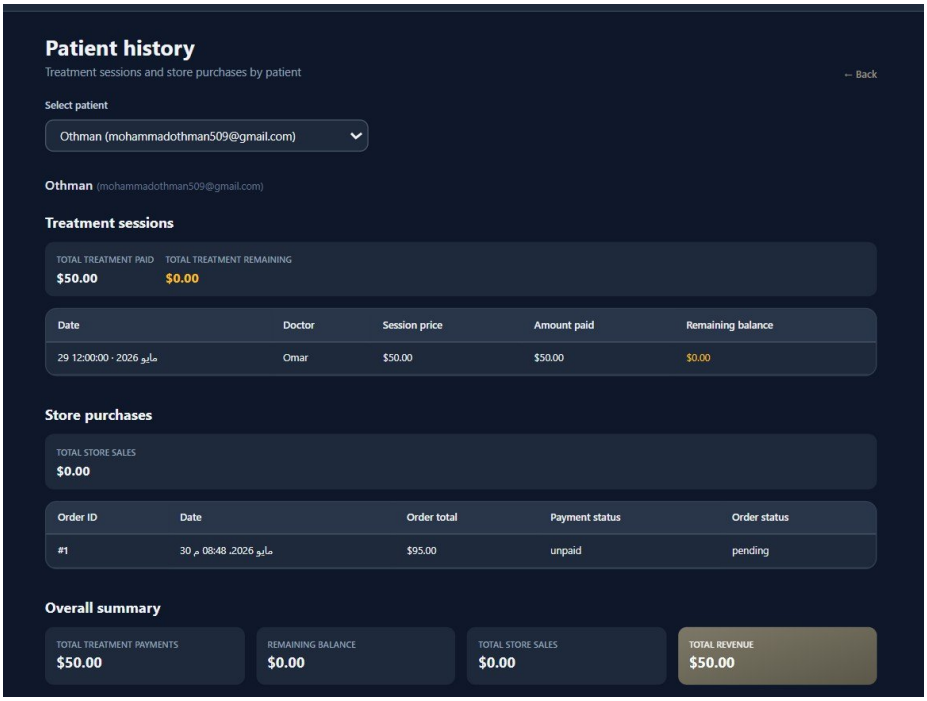

Figure 5.26: Admin patient history and financial summary interface.

Patient History (admin)

Input: the selection of a patient. **Processing:** the backend aggregates the patient's treatment sessions and store purchases. **Output:** the patient's treatment sessions, payments, and order history are displayed as an overall financial summary.

5.7 Conclusion and Recommendations

This chapter demonstrated the real implementation of the PureSkin Clinic system. The frontend was built with React and Vite, the backend with Node.js and Express exposing a REST API, and data was stored in a MySQL database. Security was implemented with JWT authentication and role-based authorization, email notifications were delivered through Nodemailer, and online payments were handled through Stripe Checkout alongside a Cash on Delivery option. The chapter linked the design-stage activity diagrams to their implementation and presented the complete role-based interface for the patient, doctor, and administrator through screenshots of the live system.

Based on the current implementation, the following improvements are recommended for future work:

- Add automated tests for the critical workflows (registration, booking, checkout, and review moderation).
- Add a password-reset feature for users who forget their credentials.
- Move uploaded product images from the local filesystem to cloud object storage for better scalability and availability.
- Introduce Redis caching for the product catalog and appointment-slot queries to reduce database load.
- Improve monitoring and logging to aid production troubleshooting.
- Add stronger production security checks, including validation that all required environment variables and secrets are set securely.

The following chapter presents the testing strategy used to validate the system's functionality and reliability.

Chapter Six: Testing

6.1 Introduction

This chapter presents the testing performed on the PureSkin Clinic system. The system was validated through manual functional and integration testing across the full stack (the React user interface, the Express REST API, and the MySQL database) rather than through an automated test suite. The goal was to confirm that the core workflows for the patient, doctor, and administrator behave as required, and that the system's security and validation rules are correctly enforced.

The testing combined manual functional testing (exercising each feature in the browser while observing API responses), integration testing of complete end-to-end flows, validation testing of input handling, and security testing of JWT authentication and role-based access control. Testing was carried out both locally and against the production deployment at pureclinic.online. It is acknowledged that the project does not yet include an automated testing framework or a continuous-integration test step; all results reported here were obtained by manual execution against a correctly configured environment.

6.2 System Testing

This section summarizes the main areas that were tested and the expected behavior derived from the implemented code. The system also enforces several built-in safety mechanisms — the appointment state machine, the completion gate, the review-eligibility check, the guarded stock decrement, and the Stripe webhook idempotency control — which are verified indirectly through the functional tests.

Authentication and authorization were tested across registration, email verification, login, and role-restricted access. Registration creates a patient account and emails a verification code; duplicate emails and invalid input are rejected, and login issues a JWT only for valid, verified credentials. Protected routes reject requests without a valid token (401), and patient, doctor, and admin routes reject the wrong role (403), with administrative actions additionally requiring the appropriate permission.

Appointment testing covered booking, slot availability, the status state machine, and the completion gate, which prevents an appointment from being marked completed without treatment evidence (a non-zero session price, a non-zero amount paid, or notes of at least three characters). The store and order workflow was tested through the checkout preview, Cash on Delivery, and Stripe checkout: the preview rejects orders that exceed available stock, the guarded stock decrement prevents overselling, and duplicate Stripe webhook deliveries are ignored. Reviews can be submitted only once and only for completed appointments, and become public only after administrator approval. Email notifications for verification and for appointment and order confirmations were also confirmed.

6.3 System Testing Plan

The following table presents the manual test cases used to validate the system, including both positive cases (a valid flow succeeds) and negative cases (invalid input or unauthorized access is rejected).

ID	Module	Test Scenario	Input Data	Expected Result
TC-01	Auth	Valid patient registration	New unique email	201; verification required, OTP emailed
TC-02	Auth	Duplicate email registration	Existing email	409 Email already in use
TC-03	Auth	Registration validation failure	Bad email / password < 6	422 validation error
TC-04	Auth	Email verification (correct OTP)	Valid email + code	200 + JWT; account activated
TC-05	Auth	Email verification (wrong OTP)	Wrong 6-digit code	400 Invalid verification code
TC-06	Auth	Login wrong password	Correct email, wrong password	401 Invalid email or password
TC-07	Auth	Login success (verified)	Valid credentials	200 + JWT; correct dashboard
TC-08	RBAC	Patient calls admin API	Patient token on /api/admin/*	403 Forbidden
TC-09	RBAC	Protected route without token	No token	401 Authentication required

ID	Module	Test Scenario	Input Data	Expected Result
TC-10	Appointments	Book appointment	Valid free slot	201 pending; slot removed from list
TC-11	Appointments	Prevent duplicate booking	Slot already booked by another patient	Slot hidden; 409 on direct API
TC-12	Appointments	Complete without treatment	status = completed, no evidence	400 completion-gate message
TC-13	Appointments	Complete with treatment	price > 0 or notes >= 3	200 completed
TC-14	Appointments	Invalid status transition	pending from confirmed	400 Invalid transition
TC-15	Store / Orders	Checkout preview over stock	qty > product.stock	400 INSUFFICIENT_STOCK
TC-16	Store / Orders	Cash on Delivery order	qty <= stock	201; stock reduced, no oversell
TC-17	Store / Orders	Stripe webhook idempotency	Duplicate event id	Second event skipped; not double-paid
TC-18	Reviews	Review non-completed appointment	Valid body, not completed	400 only completed can be reviewed
TC-19	Reviews	Duplicate review	Same appointment id	409 review already exists
TC-20	Health	Production health check	GET /api/health	200 { status: 'ok' }

6.4 Testing Plan Results

The table below records the result of executing each test case. The actual result is derived from the behavior implemented in the system and confirmed through manual execution. All test cases passed.

Test Case	Expected Result	Actual Result	Status
TC-01	201 + verification required	Account created	**Pass**
TC-02	409 conflict	Duplicate rejected	**Pass**
TC-03	422 validation	Invalid input rejected	**Pass**
TC-04	JWT after valid OTP	Account activated	**Pass**

Test Case	Expected Result	Actual Result	Status
TC-05	400 invalid code	Wrong OTP rejected	**Pass**
TC-06	401 invalid credentials	Login rejected (Fig 6.1)	**Pass**
TC-07	200 + token	Correct dashboard shown	**Pass**
TC-08	403 Forbidden	Admin API blocked	**Pass**
TC-09	401	Unauthorized	**Pass**
TC-10	201 pending appointment	Booking succeeds	**Pass**
TC-11	Duplicate prevented	Booked slot hidden (Fig 6.3)	**Pass**
TC-12	400 completion gate	Blocked without treatment	**Pass**
TC-13	200 completed	Allowed with evidence	**Pass**
TC-14	400 invalid transition	Illegal transition blocked	**Pass**
TC-15	400 insufficient stock	Preview fails	**Pass**
TC-16	Order created; stock guarded	COD succeeds, no oversell	**Pass**
TC-17	Idempotent webhook	No duplicate fulfillment	**Pass**
TC-18	400 not completed	Review rejected	**Pass**
TC-19	409 duplicate	Second review rejected	**Pass**
TC-20	200 health ok	API reachable (Fig 6.5)	**Pass**

Summary: 20 test cases were executed; all 20 passed and none failed. This confirms that the implemented workflows and the system's validation and security rules behave as expected.

6.4.1 Selected Evidence

Figure 6.1 shows the login interface rejecting invalid credentials (TC-06): a clear error message is shown and access is not granted. Figure 6.2 confirms successful email verification, where the system emails a six-digit code that the patient must enter to activate the account (TC-04).

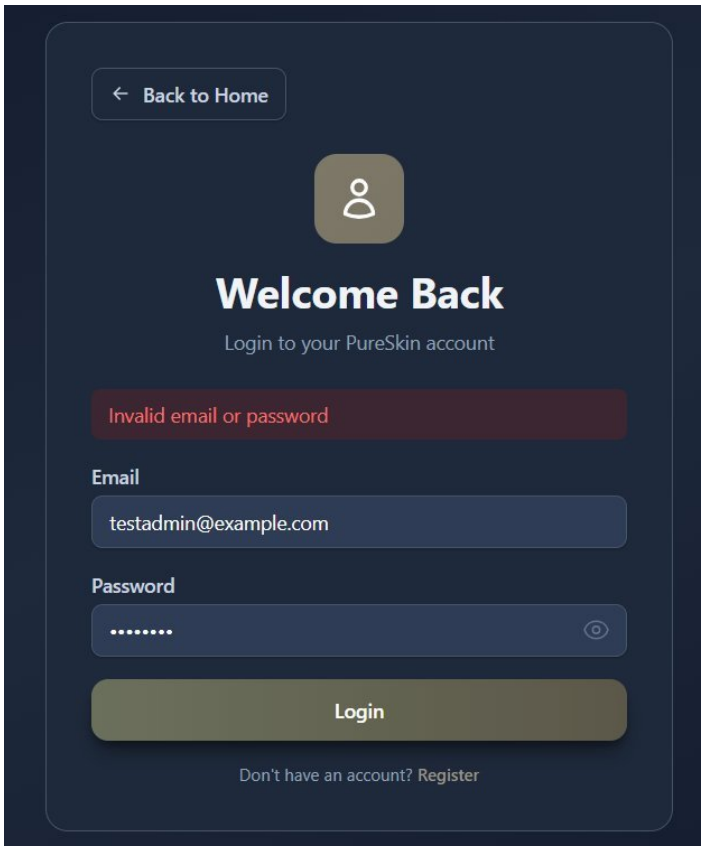

Figure 6.1: Login rejected with an 'Invalid email or password' message (TC-06).

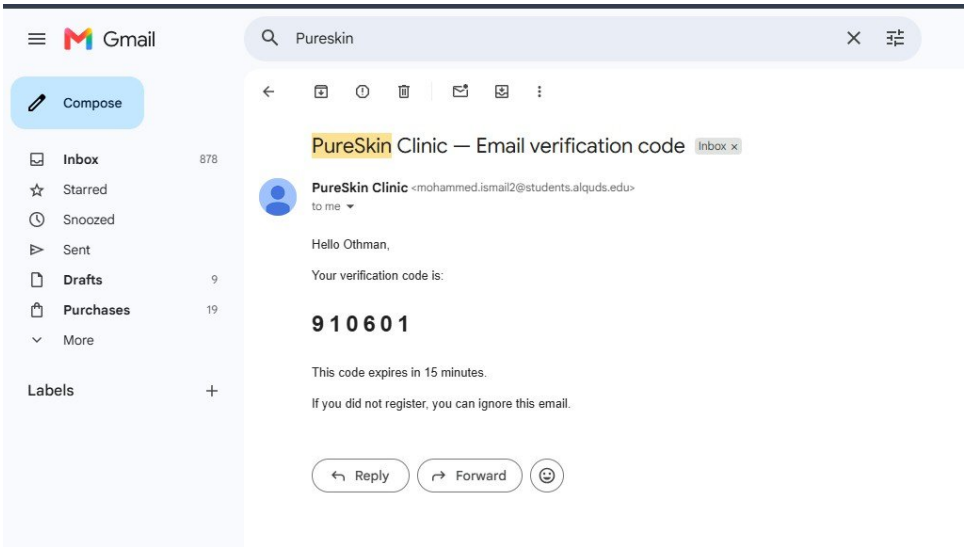

Figure 6.2: Email verification code sent during registration (TC-04).

Figure 6.3 shows the booking interface with the available time slots. This availability list is the mechanism that prevents duplicate booking (TC-11): earlier in development two patients could book the same doctor at the same time, but after the fix an active appointment (pending, confirmed, or completed) blocks the slot, while a cancelled one does not. Once Patient A books a slot, it disappears from the list shown to Patient B for the same doctor and date. For stronger protection, the backend should also return HTTP 409 with the message 'Time slot is already booked' if a duplicate request is sent directly to the API.

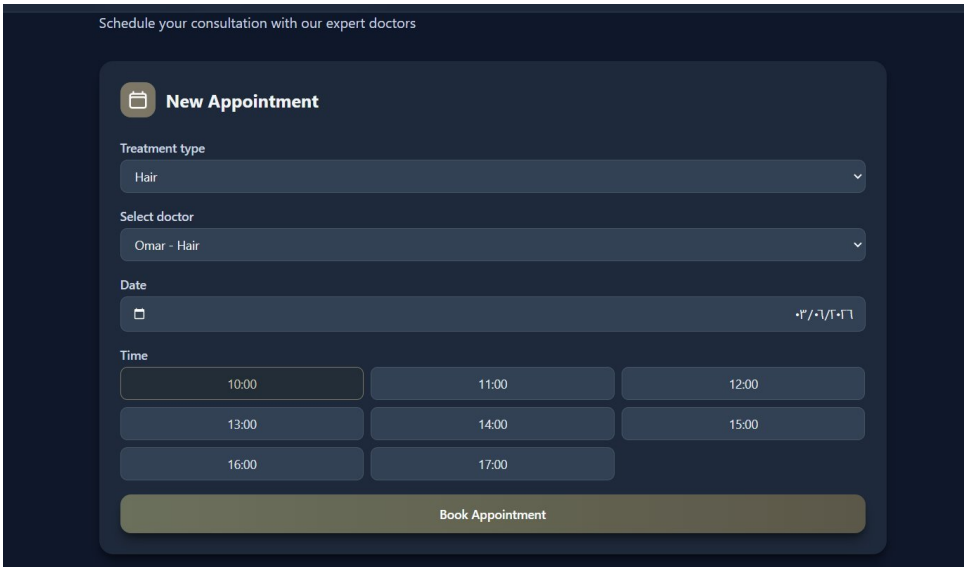

Figure 6.3: Available appointment slots used to prevent duplicate booking (TC-11).

Figure 6.4 shows the order confirmation email sent after a successful Cash on Delivery order (TC-16), and Figure 6.5 shows the production health-check endpoint returning a successful status, confirming the deployed API is reachable (TC-20).

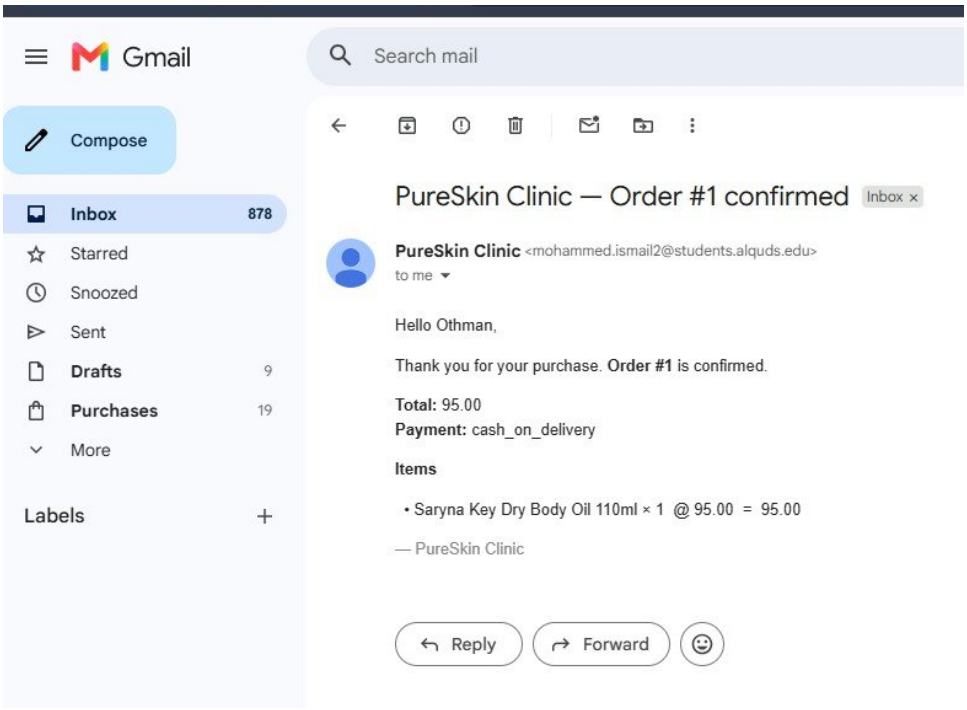

Figure 6.4: Order confirmation email after a successful order (TC-16).

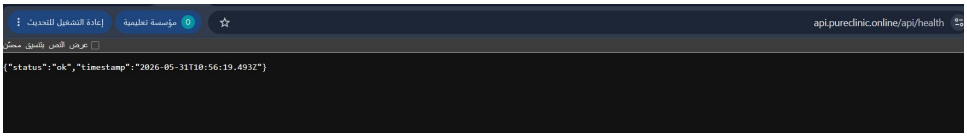

Figure 6.5: Production health-check endpoint returning { status: 'ok' } (TC-20).

6.4.2 Limitations Discovered

Testing honestly identified the following limitations, which do not affect the core feature logic but are relevant for production maturity:

- No automated regression test suite; the backend test script is a placeholder, so regressions rely on manual checks.
- No password-reset feature; a 'Forgot Password?' label exists in the interface text only.
- No global API rate limiting, except a cooldown on resending the verification code.
- The messaging module is not restricted to patient-to-doctor pairs.
- Email and Stripe flows depend on a correctly configured SMTP service and Stripe test mode, and the CI workflow deploys without running tests.

6.5 Conclusion and Recommendations

The manual and integration testing confirmed that the core workflows of PureSkin Clinic operate correctly: patients can register, verify their email, book appointments, shop and order, review, and message; doctors can manage appointments and record treatment under the completion gate; and administrators can manage users, doctors, products, and orders and moderate reviews under role-based access control. Security and validation are enforced on the server through JWT authentication, role and permission checks, input validation, the guarded stock decrement, and webhook idempotency. The main limitation is one of process: testing was manual, with no automated suite or CI test gate.

The following improvements are recommended for future work:

- Add automated unit tests for the completion gate, the stock-decrement guard, the status-transition rules, the review-eligibility check, and the webhook idempotency control.
- Add API integration tests for the authentication, appointment, and order flows against a dedicated test database.
- Add end-to-end user-interface tests for the main register, book, and checkout paths.
- Add a continuous-integration step that runs the tests on every push before deployment.
- Add load and security testing, including rate limiting on login and a password-reset flow.

With these additions, the system's reliability could be maintained automatically as it evolves, reducing the dependence on the manual testing demonstrated in this chapter.

Chapter Seven: Conclusion

7.1 Conclusion

This project set out to design and implement PureSkin Clinic, a full-stack, role-based web application that unifies the core operations of a dermatology and aesthetic clinic into a single platform. The work progressed through clearly defined stages — system analysis, requirements specification, system design, implementation, and testing — each documented in the preceding chapters.

The system analysis established the feasibility of the project and the value of replacing fragmented, manual workflows with an integrated digital solution. The requirements chapter defined the functional and non-functional requirements across the patient, doctor, and administrator roles, while the design chapter translated these requirements into a concrete architecture expressed through class, sequence, entity relationship, and activity diagrams. The implementation chapter then realized this design as a working system built with a React frontend, a Node.js and Express REST API, and a MySQL database, secured with JWT authentication and role-based access control, and integrated with an SMTP email service and Stripe Checkout.

The most significant outcome of this project is that the system is not merely a prototype but a fully implemented and publicly deployed application, live at https://pureclinic.online with its API at https://api.pureclinic.online/api. The testing chapter confirmed, through manual functional and integration testing, that the core workflows operate correctly and that the system's security and validation mechanisms — including the appointment completion gate, the guarded stock decrement, and the Stripe webhook idempotency control — behave as intended.

In summary, PureSkin Clinic successfully demonstrates how modern web technologies can be combined to deliver a secure, integrated, and practical clinic management and e-commerce platform. The project achieved the objectives defined at its outset and provides a solid foundation for further enhancement. The main limitation identified is the absence of an automated test suite, and the following improvements are recommended for future work:

- Introduce an automated testing framework with unit, integration, and end-to-end tests, executed automatically through a continuous-integration pipeline on every change.

- Add a secure password-reset feature and strengthen production security with rate limiting and validation of environment variables at startup.
- Migrate uploaded product images to cloud object storage and introduce a caching layer to improve scalability and performance under higher load.
- Extend the platform with native mobile applications and explore multi-clinic (multi-tenant) support to broaden its applicability.

Through the development of PureSkin Clinic, the team gained substantial practical experience in full-stack web development, database design, secure authentication, third-party service integration, and production deployment — experience that fulfills the academic and engineering goals of this graduation project.

References

[1] Sandhu, R. S., & Samarati, P. (1994). Access control: Principles and practice. IEEE Communications Magazine, 32(9), 40–48.

[2] Shortliffe, E. H., & Cimino, J. J. (2014). Biomedical Informatics: Computer Applications in Health Care and Biomedicine. Springer.

[3] Ramakrishnan, R., & Gehrke, J. (2002). Database Management Systems. McGraw-Hill.

[4] Kruse, C. S., Smith, B., Vanderlinden, H., & Nealand, A. (2017). Security techniques for the electronic health records. Journal of Medical Systems, 41(8), 127.

[5] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.

[6] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT), RFC 7519. Internet Engineering Task Force (IETF).

[7] React Documentation. Meta Open Source. Retrieved from https://react.dev

[8] Express.js Documentation. OpenJS Foundation. Retrieved from https://expressjs.com

[9] MySQL 8.0 Reference Manual. Oracle Corporation. Retrieved from https://dev.mysql.com/doc

[10] Stripe API Documentation. Stripe, Inc. Retrieved from https://stripe.com/docs